SNOWFLAKE

Retail Data Ingestion & Data Quality Lab

Technical implementation record

Platform	Snowflake
Database	SASI_RA_DB
Schema	RAW_INGESTION
Warehouse	SASI_RETAIL_INGESTION_WH (X-Small)

Version 3 – incorporates the original ingestion and profiling evidence plus the next 14 screenshots supplied on 05 November 2026. This living document records staging transformations, analytical queries and an initial query-profile comparison.

1. Executive summary

This lab documents an end-to-end Snowflake retail-data workflow: provisioning, CSV staging, COPY INTO ingestion, raw-layer validation, data-quality profiling, typed staging transformations and business-oriented analytical queries. The raw layer contains 125 customer rows and 213 order rows. The later staging results contain 121 customers and 198 orders, showing that the transformation pipeline does not simply mirror raw row counts.

The latest evidence extends the lab from ingestion into analytics. The staged customer table has 121 rows (121 distinct customer IDs), 3 missing emails and no NULL signup dates. The staged orders table has 198 rows. Revenue-eligible analytical outputs report total revenue of 189,719.06 across six product categories and across ten months from January to October 2024. The screenshots also compare an original and a CTE-based city-sales query; the small dataset and query-profile measurements do not establish a durable performance improvement.

2. Objectives and scope

- Provision a minimal Snowflake workspace for a retail ingestion exercise.
- Land customer and order CSV files in raw tables without prematurely transforming source values.
- Validate ingestion using COPY INTO results and table-level row counts.
- Profile data quality before designing transformations or an analytical model.
- Document the teardown commands for the lab-created database and warehouse.

3. Environment and object inventory

Object type	Object name	Purpose
Database	SASI_RA_DB	Container for the lab's ingestion objects.
Schema	RAW_INGESTION	Raw landing area for source files and tables.
Warehouse	SASI_RETAIL_INGESTION_WH	X-Small compute resource for SQL execution and loading.
Internal stage	ECOMMERCE_INTERNAL_STAGE	Stores uploaded customer and order CSV files.
File format	CUSTOMERS_CSV_FORMAT	CSV parsing configuration for customer records.
File format	ORDERS_CSV_FORMAT	CSV parsing configuration for order records.
Table	RAW_CUSTOMERS	Raw customer source data.
Table	RAW_ORDERS	Raw order/transaction source data.

4. Session and resource provisioning

The worksheet first inspects the active Snowflake session using CURRENT_USER(), CURRENT_ROLE(), CURRENT_WAREHOUSE(), CURRENT_DATABASE() and CURRENT_SCHEMA(). The captured session reports user SEBBY, role ACCOUNTADMIN, warehouse SASI_RETAIL_INGESTION_WH, database SASI_RA_DB and schema RAW_INGESTION.

The database and schema are created if absent. The warehouse is configured as X-Small, with automatic suspension after 60 seconds, automatic resume enabled, and initially suspended.

```
USE ROLE ACCOUNTADMIN;

CREATE DATABASE IF NOT EXISTS SASI_RA_DB;
CREATE SCHEMA IF NOT EXISTS SASI_RA_DB.RAW_INGESTION;

CREATE WAREHOUSE IF NOT EXISTS SASI_RETAIL_INGESTION_WH
  WAREHOUSE_SIZE = 'XSMALL'
```

```
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

SHOW DATABASES LIKE 'SASI_RA_DB';
SHOW SCHEMAS LIKE 'RAW_INGESTION' IN DATABASE SASI_RA_DB;
SHOW WAREHOUSES LIKE 'SASI_RETAIL_INGESTION_WH';
USE WAREHOUSE SASI_RETAIL_INGESTION_WH;
```

Learning note: a virtual warehouse supplies compute; it does not store the database tables. AUTO_SUSPEND helps reduce idle compute consumption, while AUTO_RESUME allows queries to start the warehouse when needed.

5. CSV file formats

Two named CSV file formats are created in RAW_INGESTION. The screenshots show comma-delimited fields, newline record delimiters, one skipped header row, double-quote optional enclosure, empty fields interpreted as NULL, UTF-8 encoding, and column-count mismatch errors enabled.

```
CREATE OR REPLACE FILE FORMAT
  SASI_RA_DB.RAW_INGESTION.CUSTOMERS_CSV_FORMAT
  TYPE = CSV
  FIELD_DELIMITER = ','
  RECORD_DELIMITER = '\\n'
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  EMPTY_FIELD_AS_NULL = TRUE
  TRIM_SPACE = FALSE
  ENCODING = 'UTF8'
  ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE;

CREATE OR REPLACE FILE FORMAT
  SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT
  TYPE = CSV
  FIELD_DELIMITER = ','
  RECORD_DELIMITER = '\\n'
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  EMPTY_FIELD_AS_NULL = TRUE
  TRIM_SPACE = FALSE
  ENCODING = 'UTF8'
  ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE;

SHOW FILE FORMATS IN SCHEMA SASI_RA_DB.RAW_INGESTION;
```

Why use named formats? They centralize parsing behavior and make COPY INTO statements shorter and more repeatable. The customer and order formats are currently identical in the captured configuration, but are separate objects so they can evolve independently.

6. Internal stage and staged files

An internal named stage called ECOMMERCE_INTERNAL_STAGE is created in RAW_INGESTION with directory-table support enabled. The LIST command confirms two files are present.

```
CREATE OR REPLACE STAGE
  SASI_RA_DB.RAW_INGESTION.ECOMMERCE_INTERNAL_STAGE
  DIRECTORY = (ENABLE = TRUE);

SHOW STAGES IN SCHEMA SASI_RA_DB.RAW_INGESTION;

LIST @SASI_RA_DB.RAW_INGESTION.ECOMMERCE_INTERNAL_STAGE;
```

Staged file	Size (bytes)	MD5 shown in worksheet
ecommerce_internal_stage/messy_customers.csv	8,768	5d754ffd43e80240154eed4e77e31a0c
ecommerce_internal_stage/messy_ecommerce_orders.csv	18,560	9046dbfcf3bdde9c53030fcefa44e045

The LIST output verifies that Snowflake can see the staged objects and reports their sizes and checksums. The staged object names include the directory prefix shown by the stage listing; the COPY statements use the filenames relative to the stage path.

7. Raw landing tables

Two raw tables are created. The customer table has five VARCHAR columns: CUSTOMER_ID, CUSTOMER_NAME, EMAIL, SIGNUP_DATE and CUSTOMER_CITY. The order table has eleven VARCHAR columns: ORDER_ID, CUSTOMER_ID, PURCHASE_DATE, DELIVERY_DATE, PRODUCT_CATEGORY, PRICE, QUANTITY, PAYMENT_TYPE, CUSTOMER_CITY, REVIEW_SCORE and REVIEW_COMMENT.

```
CREATE OR REPLACE TABLE SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS (
  CUSTOMER_ID   VARCHAR(100),
  CUSTOMER_NAME VARCHAR(255),
  EMAIL         VARCHAR(255),
  SIGNUP_DATE   VARCHAR(100),
  CUSTOMER_CITY VARCHAR(255)
);

CREATE OR REPLACE TABLE SASI_RA_DB.RAW_INGESTION.RAW_ORDERS (
  ORDER_ID       VARCHAR(100),
  CUSTOMER_ID    VARCHAR(100),
  PURCHASE_DATE  VARCHAR(100),
  DELIVERY_DATE  VARCHAR(100),
  PRODUCT_CATEGORY VARCHAR(100),
  PRICE          VARCHAR(100),
  QUANTITY       VARCHAR(100),
  PAYMENT_TYPE   VARCHAR(100),
  CUSTOMER_CITY  VARCHAR(255),
  REVIEW_SCORE   VARCHAR(100),
  REVIEW_COMMENT VARCHAR(2000)
);

DESC TABLE SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS;
DESC TABLE SASI_RA_DB.RAW_INGESTION.RAW_ORDERS;
```

Design rationale: keeping raw columns as VARCHAR preserves the source representation during ingestion. Type conversion, date normalization, numeric validation and standardization can be handled in a later staging/transformation step. This is useful for messy input because values such as '$60.05', '--', '?', mixed date strings and negative quantities are not rejected simply because they do not yet match a strict target type.

8. Loading data with COPY INTO

The screenshots show separate COPY INTO commands for customers and orders. Each command selects a named file from the internal stage, references its corresponding named file format and uses ABORT_STATEMENT as the error policy.

```
COPY INTO SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS
FROM @SASI_RA_DB.RAW_INGESTION.ECOMMERCE_INTERNAL_STAGE
FILES = ('messy_customers.csv')
FILE_FORMAT = (
  FORMAT_NAME = 'SASI_RA_DB.RAW_INGESTION.CUSTOMERS_CSV_FORMAT'
)
ON_ERROR = 'ABORT_STATEMENT';
```

```
COPY INTO SASI_RA_DB.RAW_INGESTION.RAW_ORDERS
FROM @SASI_RA_DB.RAW_INGESTION.ECOMMERCE_INTERNAL_STAGE
FILES = ('messy_ecommerce_orders.csv')
FILE_FORMAT = (
  FORMAT_NAME = 'SASI_RA_DB.RAW_INGESTION.ORDERS_CSV_FORMAT'
)
ON_ERROR = 'ABORT_STATEMENT';
```

Observed result: the visible COPY result reports status LOADED, 213 rows parsed, 213 rows loaded, and zero errors. The screenshots establish this result for the order-file load. The separate table count query then reports 125 customer records and 213 order records.

9. Ingestion validation and sample inspection

```
SELECT 'RAW_CUSTOMERS' AS TABLE_NAME, COUNT(*) AS TOTAL_RECORDS
FROM SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS
UNION ALL
SELECT 'RAW_ORDERS', COUNT(*)
FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS;

SELECT * FROM SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS LIMIT 10;
SELECT * FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS LIMIT 10;
```

Raw table	Observed row count	Validation
RAW_CUSTOMERS	125	Count query returned 125
RAW_ORDERS	213	Count query returned 213

The LIMIT 10 queries are used for a quick visual inspection of loaded values. The screenshots reveal inconsistent city capitalization (for example, 'Rio de Janeiro', 'rio de janeiro' and uppercase variants), mixed date formats, different payment-type spellings, null review fields, and non-numeric placeholders in price.

10. Initial data-quality profiling

10.1 NULL profiling

COUNT_IF(column IS NULL) is used to count nulls per column. The captured customer profiling result reports 125 total rows and 3 missing EMAIL values; the other listed customer fields have zero NULLs. The order profiling result reports 213 total rows and the visible counts below.

Column / check	Missing count	Interpretation
Customers: EMAIL	3	Three customer records have NULL email.
Orders: ORDER_ID	3	Three order rows have NULL order identifiers.
Orders: CUSTOMER_ID	3	Three order rows have NULL customer identifiers.
Orders: PURCHASE_DATE	3	Three order rows have NULL purchase dates.
Orders: DELIVERY_DATE	18	Delivery date is the largest visible NULL issue.
Orders: PRODUCT_CATEGORY	3	Three category values are NULL.
Orders: PRICE	4	Four price values are NULL.
Orders: QUANTITY	3	Three quantity values are NULL.
Orders: PAYMENT_TYPE	3	Three payment type values are NULL.

10.2 Empty-string profiling

A separate check uses TRIM(column) = '' to identify empty or whitespace-only strings. The displayed results report zero empty EMAIL, CUSTOMER_NAME and CUSTOMER_CITY values in RAW_CUSTOMERS, and zero empty PRICE, QUANTITY and ORDER_ID values in RAW_ORDERS. This is distinct from NULL profiling: a NULL value and an empty string are not the same condition.

```
SELECT
  COUNT_IF(TRIM(EMAIL) = '') AS EMPTY_EMAIL,
  COUNT_IF(TRIM(CUSTOMER_NAME) = '') AS EMPTY_CUSTOMER_NAME,
  COUNT_IF(TRIM(CUSTOMER_CITY) = '') AS EMPTY_CUSTOMER_CITY
FROM RAW_CUSTOMERS;

SELECT
  COUNT_IF(TRIM(PRICE) = '') AS EMPTY_PRICE,
  COUNT_IF(TRIM(QUANTITY) = '') AS EMPTY_QUANTITY,
  COUNT_IF(TRIM(ORDER_ID) = '') AS EMPTY_ORDER_ID
FROM RAW_ORDERS;
```

10.3 Duplicate detection

The lab checks repeated identifiers with GROUP BY, HAVING COUNT(*) > 1, and descending occurrence counts. The customer identifier check returns four repeated CUSTOMER_ID values: CUST286, CUST200, CUST197 and CUST250, each occurring twice in the captured output.

```
SELECT CUSTOMER_ID, COUNT(*) AS OCCURRENCES
FROM RAW_CUSTOMERS
WHERE CUSTOMER_ID IS NOT NULL
GROUP BY CUSTOMER_ID
HAVING COUNT(*) > 1
ORDER BY OCCURRENCES DESC;

SELECT ORDER_ID, COUNT(*) AS OCCURRENCES
FROM RAW_ORDERS
WHERE ORDER_ID IS NOT NULL
GROUP BY ORDER_ID
HAVING COUNT(*) > 1
ORDER BY OCCURRENCES DESC;
```

The order identifier check displays 13 rows with repeated ORDER_ID values in the screenshot; the visible occurrence count is 2. A subsequent full-row grouping query is also run to identify exact repeated records across all selected columns. The screenshot shows 14 result rows for that full-row duplicate query, including one row where the displayed values are NULL. These checks are exploratory: repeated IDs do not automatically prove that records should be deleted. Business rules must determine whether the rows are duplicate transactions, legitimate line items, or data-entry issues.

```
-- Example full-row duplicate check for customers
SELECT
  CUSTOMER_ID, CUSTOMER_NAME, EMAIL, SIGNUP_DATE, CUSTOMER_CITY,
  COUNT(*) AS DUPLICATE_COUNT
FROM RAW_CUSTOMERS
GROUP BY CUSTOMER_ID, CUSTOMER_NAME, EMAIL, SIGNUP_DATE, CUSTOMER_CITY
HAVING COUNT(*) > 1;

-- Example full-row duplicate check for orders
SELECT
  ORDER_ID, CUSTOMER_ID, PURCHASE_DATE, DELIVERY_DATE, PRODUCT_CATEGORY,
  PRICE, QUANTITY, PAYMENT_TYPE, CUSTOMER_CITY, REVIEW_SCORE, REVIEW_COMMENT,
  COUNT(*) AS DUPLICATE_COUNT
FROM RAW_ORDERS
GROUP BY ALL
HAVING COUNT(*) > 1;
```

11. Business and technical observations

- The raw landing layer is functioning: source files are visible in the stage and the load result reports successful ingestion.
- The order dataset contains 213 rows and the customer dataset contains 125 rows.
- The source data is intentionally messy and requires standardization before trusted reporting.
- NULL profiling and empty-string profiling are separate checks and should both be retained in a data-quality checklist.
- Duplicate identifiers should be investigated before deduplication; blindly deleting repeated IDs could remove valid business records.
- Dates and numeric values should be parsed with defensive logic because the samples include multiple date formats, currency symbols/placeholders and negative quantities.
- The next logical phase is a small cleaned/staging layer followed by a few business-reasoning queries and straightforward joins, without introducing a formal star schema unless the task later requires it.

12. Cleanup and resource teardown

The cleanup step is intended to remove only the lab-created database and warehouse. Dropping the database removes its schemas, tables, internal stage, file formats and other contained objects. Dropping the warehouse removes the lab compute resource. The screenshots supplied so far do not show the cleanup commands being executed or the verification results, so the commands below are documented as the teardown procedure, not as a confirmed outcome.

```
USE ROLE ACCOUNTADMIN;

-- Optional explicit suspension before removal
ALTER WAREHOUSE IF EXISTS SASI_RETAIL_INGESTION_WH SUSPEND;

-- Removes RAW_INGESTION and all objects inside SASI_RA_DB
DROP DATABASE IF EXISTS SASI_RA_DB;

-- Removes the lab virtual warehouse
DROP WAREHOUSE IF EXISTS SASI_RETAIL_INGESTION_WH;

-- Verify that the lab objects are gone
SHOW DATABASES LIKE 'SASI_RA_DB';
SHOW WAREHOUSES LIKE 'SASI_RETAIL_INGESTION_WH';
```

Expected verification: the SHOW commands should return no matching database or warehouse. This procedure does not delete the Snowflake account, user, role, worksheet, local source CSV files or unrelated databases and warehouses. Before executing, confirm that no other project depends on these specifically named objects.

13. Current status after revision 3

Area	Current documentation status	Evidence / next step
Provisioning and raw ingestion	Documented from screenshots	Database, schema, warehouse, formats, stage and tables.
COPY INTO and counts	Documented from screenshots	213 order rows loaded; 125 customers and 213 orders counted.
Initial data profiling	Documented from screenshots	NULL, empty-string and duplicate checks captured.
Cleanup execution	Procedure documented; outcome unverified	Add screenshot of DROP statements and empty SHOW results.
Further analytical queries / joins	Pending	Add screenshots and business interpretation when supplied.

Appendix A. Screenshot evidence

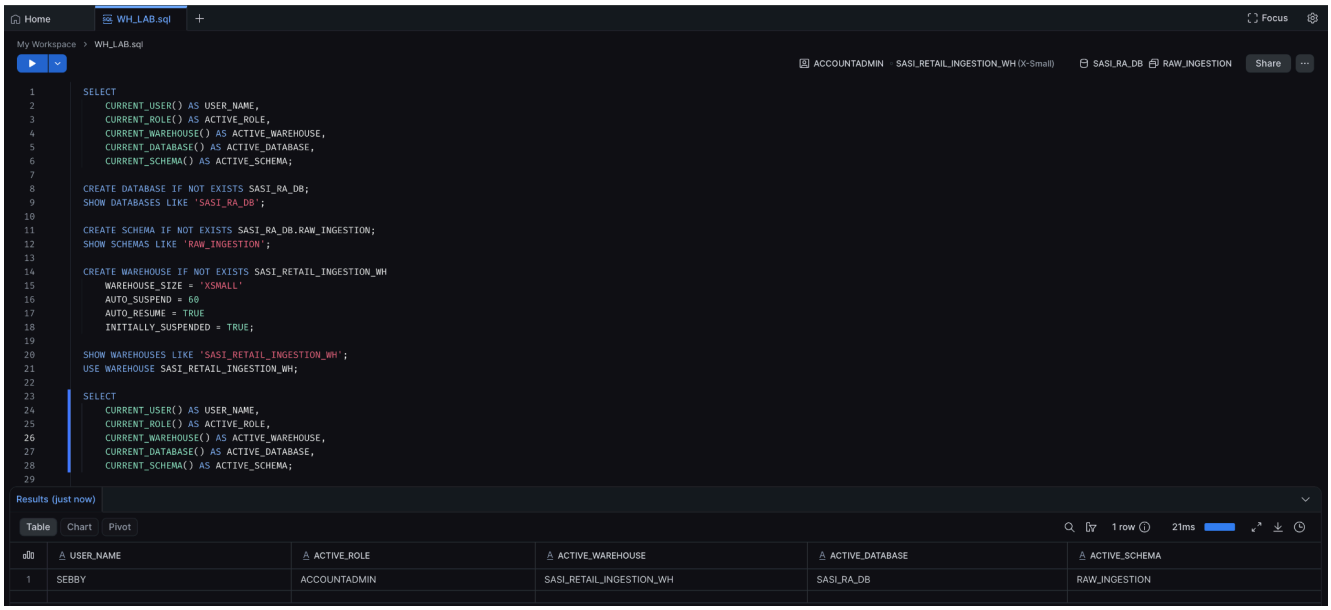
The screenshots below are embedded in the order supplied. Captions identify the main activity visible in each image. As more screenshots arrive, this appendix and the relevant procedure sections will be extended.

Appendix A. Screenshot evidence

The screenshots below are embedded in the order supplied. Captions identify the main activity visible in each image. As more screenshots arrive, this appendix and the relevant procedure sections will be extended.

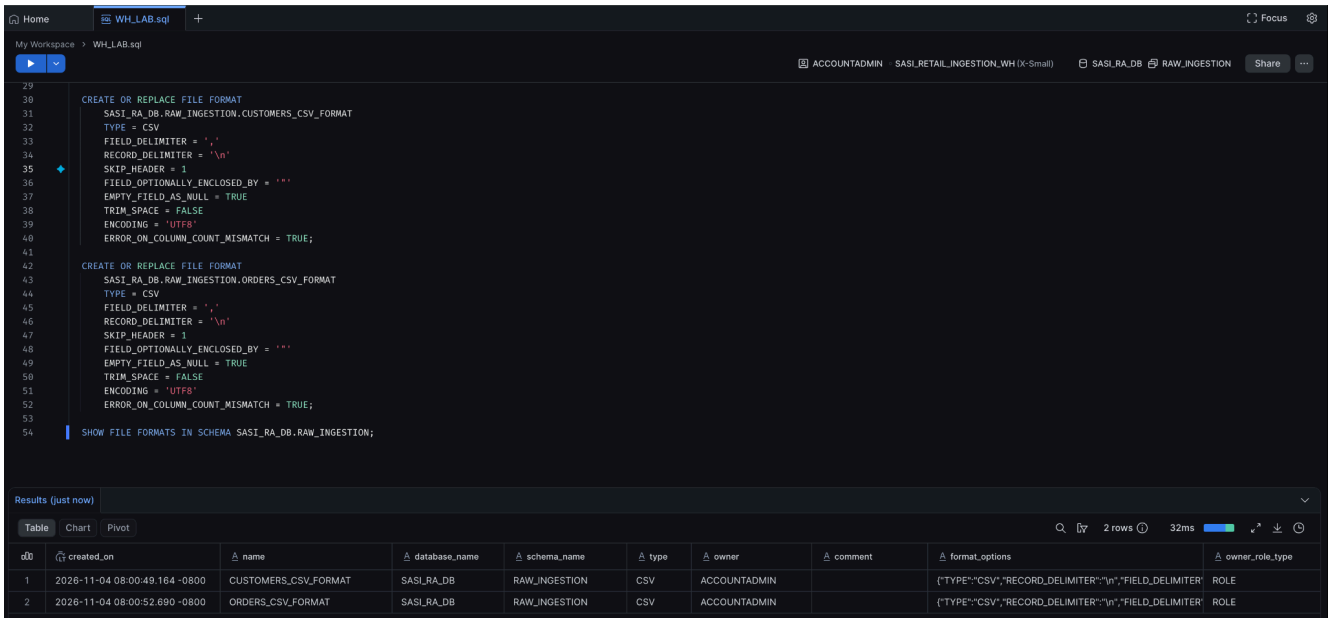
Evidence 01

Session context and initial database/schema/warehouse setup.



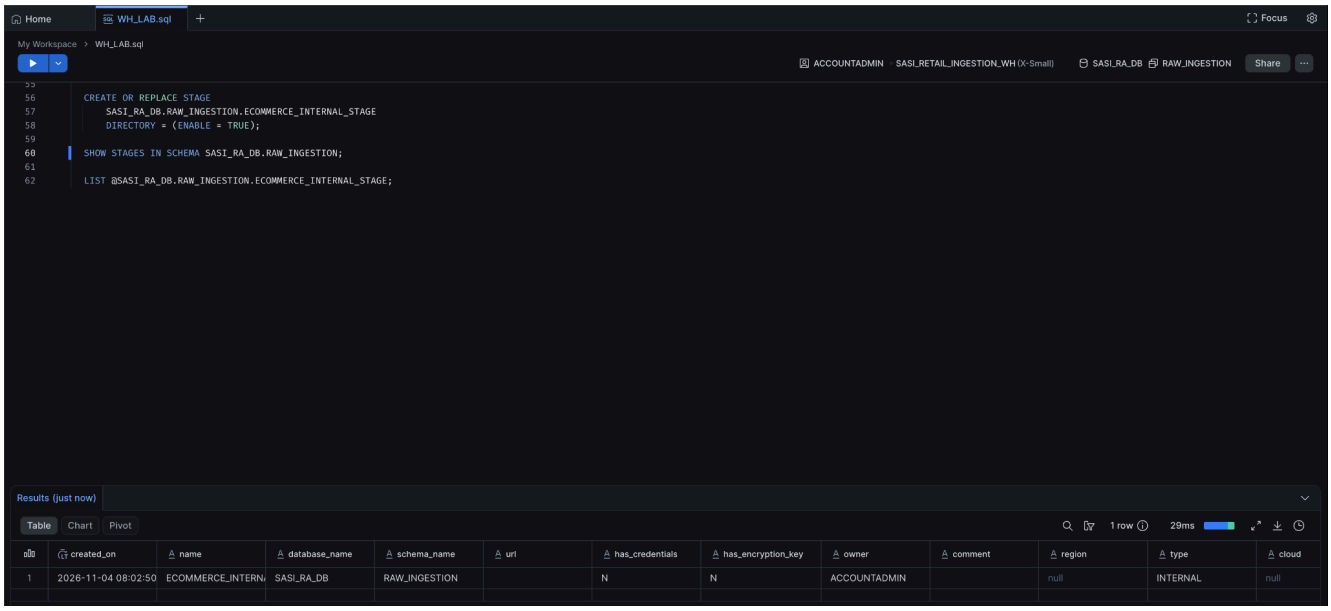
Evidence 02

Customer and order CSV file-format definitions and SHOW FILE FORMATS output.



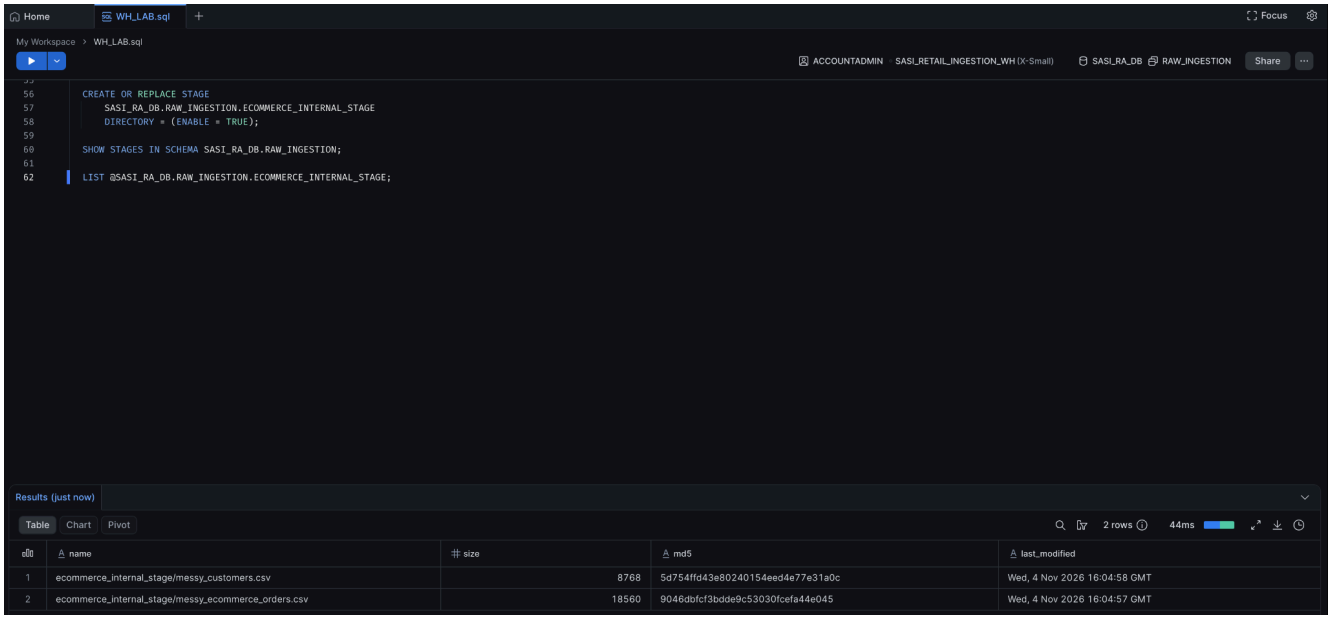
Evidence 03

Internal stage creation and stage listing.



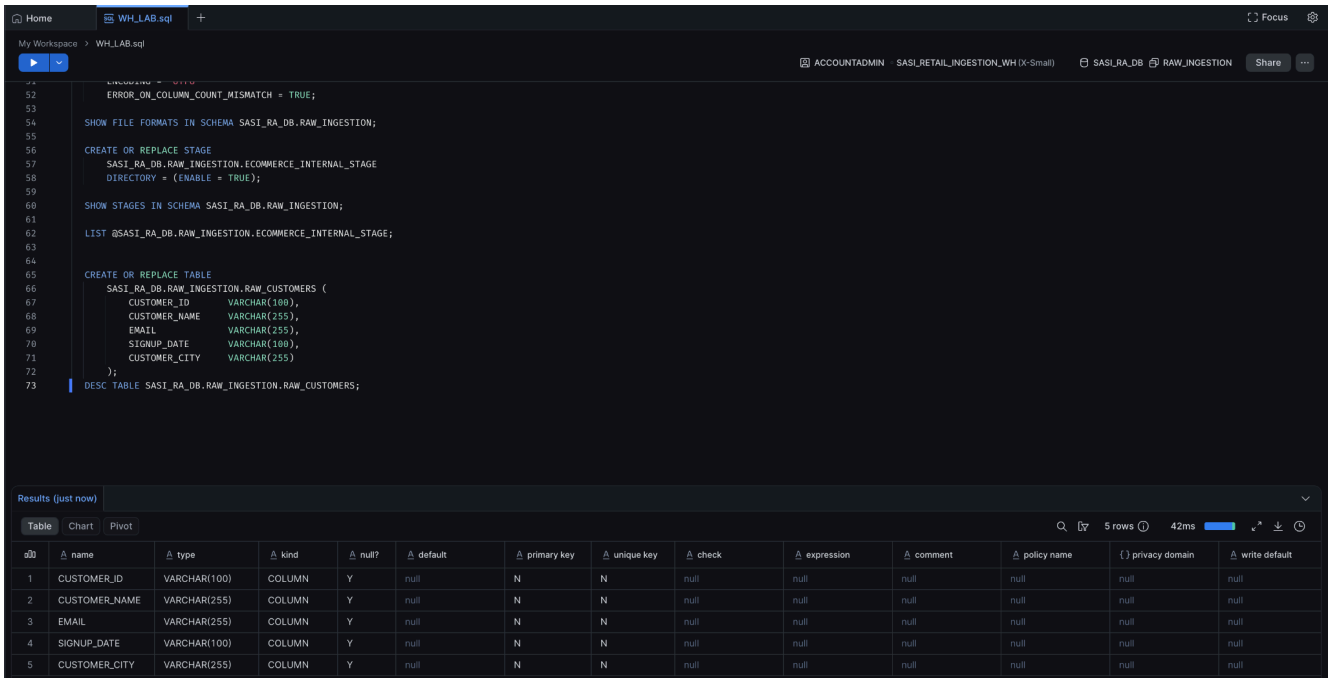
Evidence 04

Stage file listing showing the two staged CSV files and their metadata.



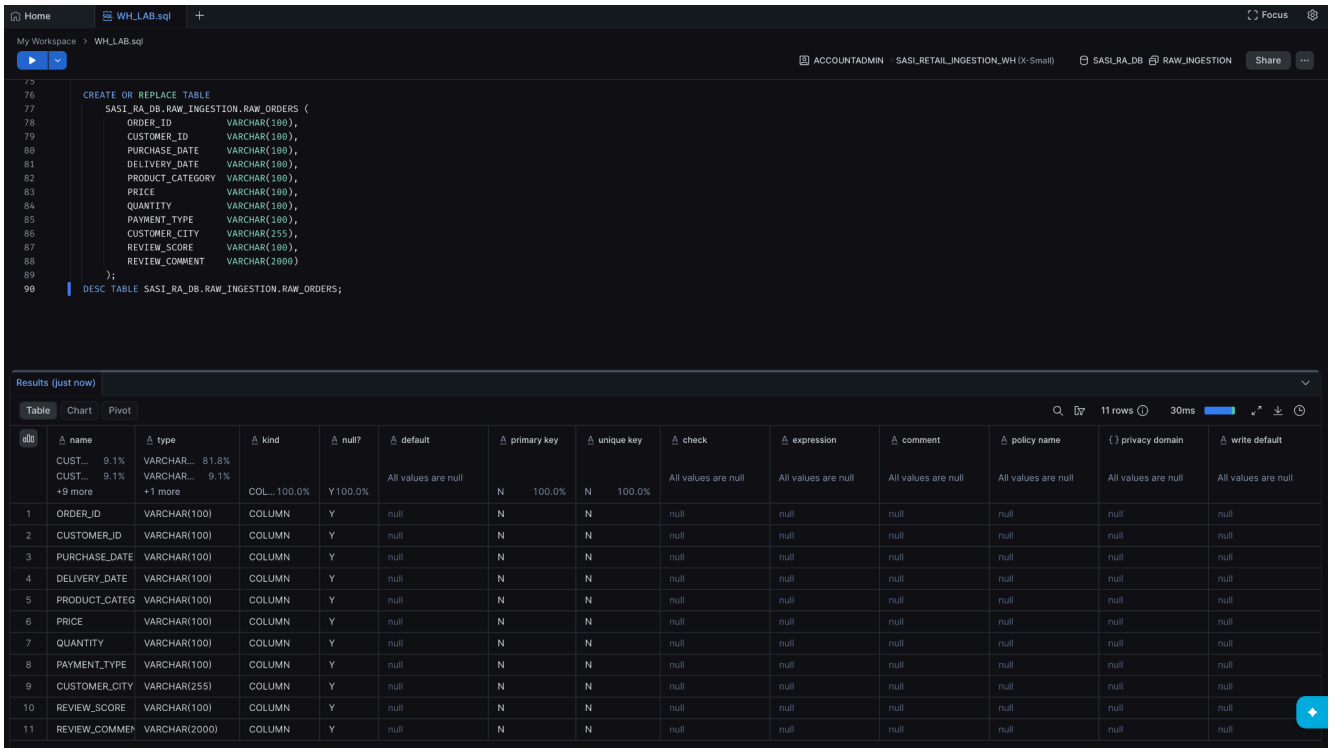
Evidence 05

RAW_CUSTOMERS table definition and DESCRIBE output.



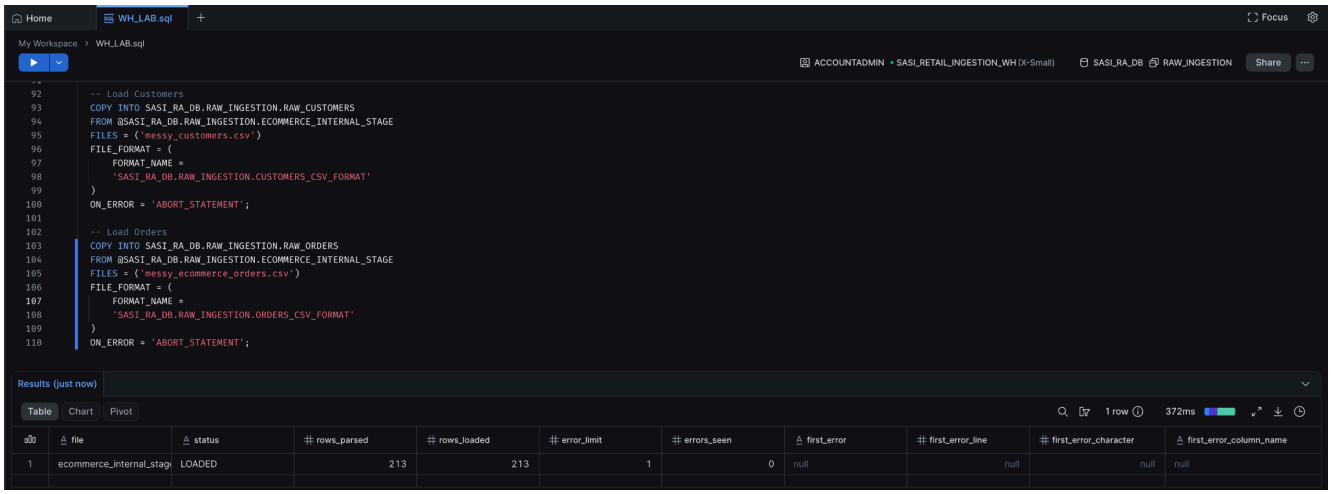
Evidence 06

RAW_ORDERS table definition and DESCRIBE output.



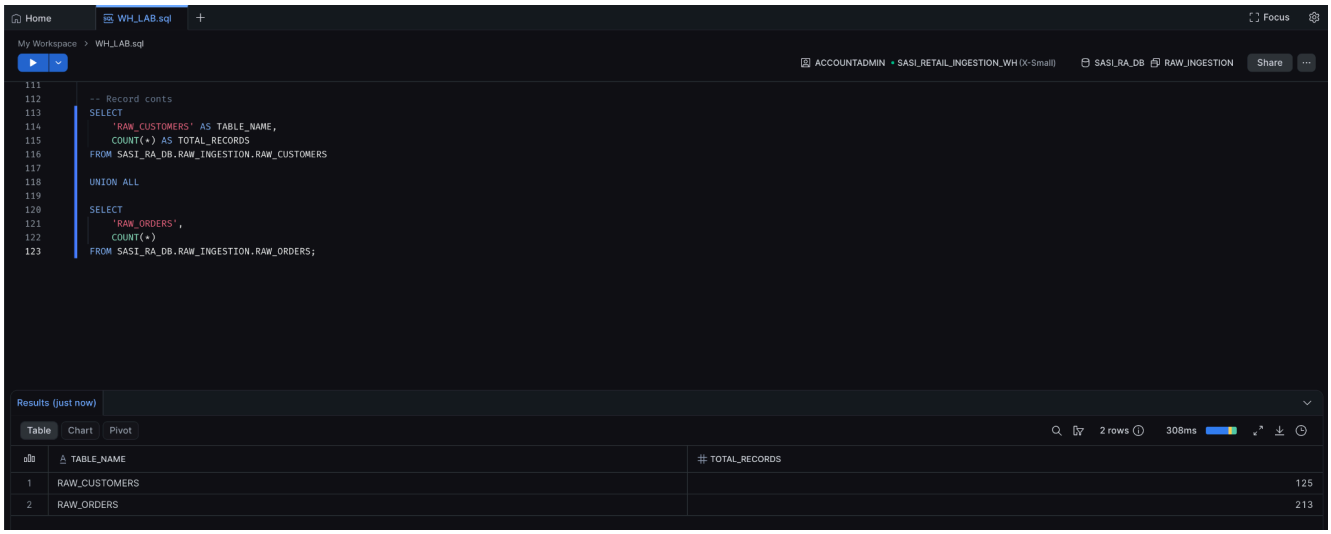
Evidence 07

COPY INTO load statements for the customer and order files.



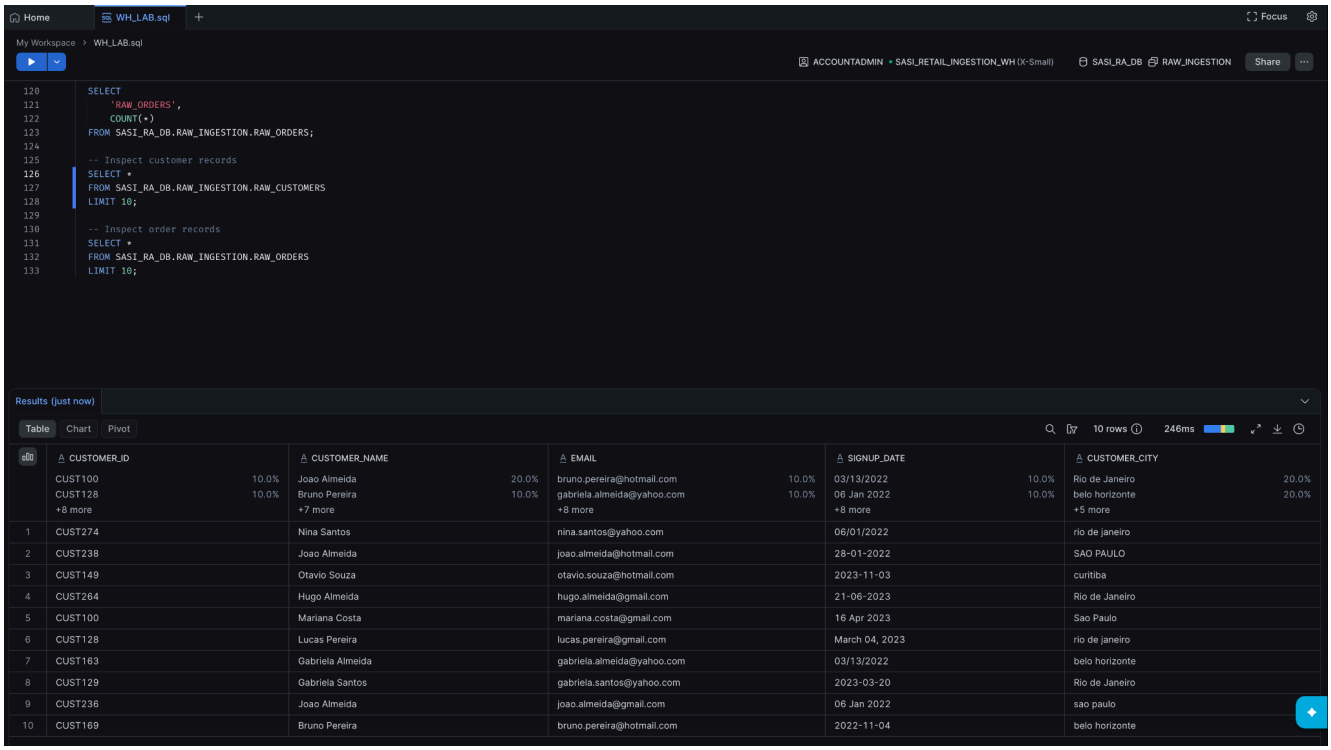
Evidence 08

Row-count validation: 125 customers and 213 orders.



Evidence 09

Sample customer rows from RAW_CUSTOMERS.



```
SELECT
    'RAW_ORDERS',
    COUNT(*)
FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS;

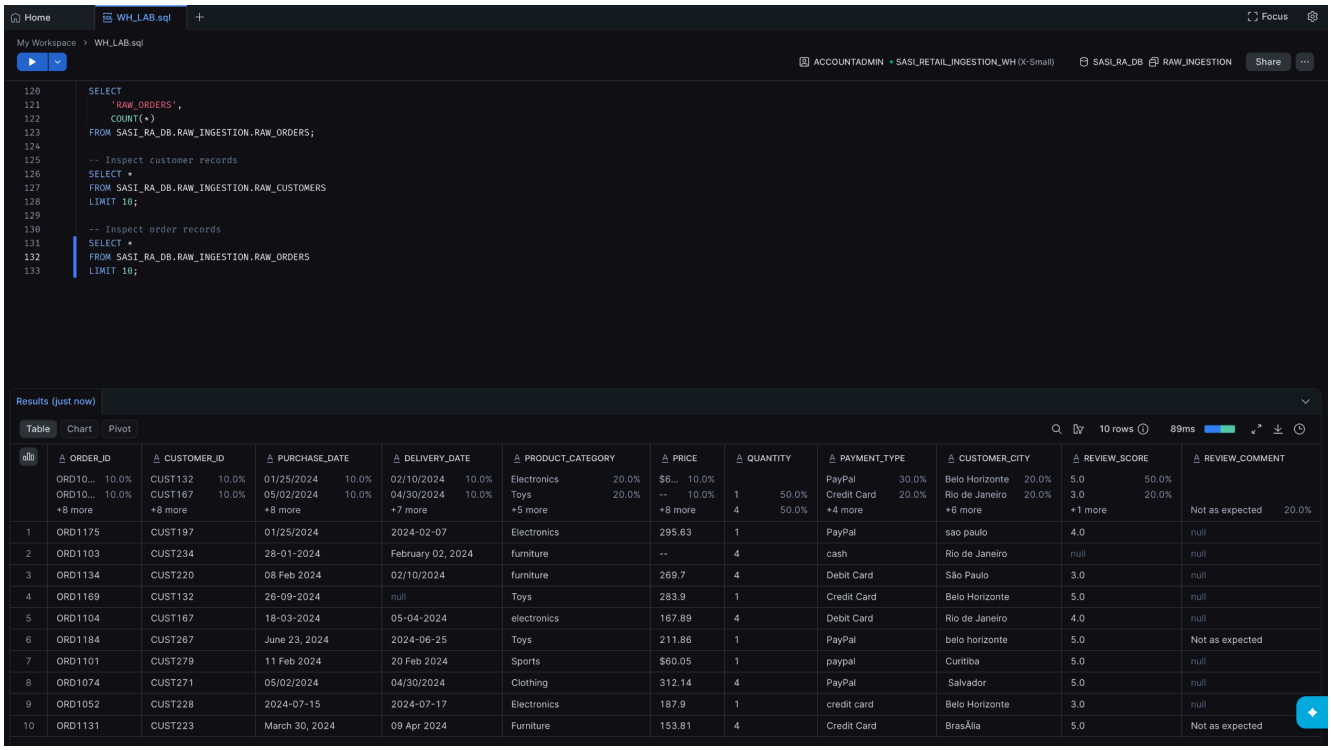
-- Inspect customer records
SELECT *
FROM SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS
LIMIT 10;

-- Inspect order records
SELECT *
FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS
LIMIT 10;
```

	CUSTOMER_ID	CUSTOMER_NAME	EMAIL	SIGNUP_DATE	CUSTOMER_CITY
	CUST100 10.0% CUST128 10.0% +8 more	Joao Almeida 20.0% Bruno Pereira 10.0% +7 more	bruno.pereira@hotmail.com 10.0% gabriela.almeida@yahoo.com 10.0% +8 more	03/13/2022 10.0% 06 Jan 2022 10.0% +8 more	Rio de Janeiro 20.0% belo horizonte 20.0% +5 more
1	CUST274	Nina Santos	nina.santos@yahoo.com	06/01/2022	rio de janeiro
2	CUST238	Joao Almeida	joao.almeida@hotmail.com	28-01-2022	SAO PAULO
3	CUST149	Otavio Souza	otavio.souza@hotmail.com	2023-11-03	curitiba
4	CUST264	Hugo Almeida	hugo.almeida@gmail.com	21-06-2023	Rio de Janeiro
5	CUST100	Mariana Costa	mariana.costa@gmail.com	16 Apr 2023	Sao Paulo
6	CUST128	Lucas Pereira	lucas.pereira@gmail.com	March 04, 2023	rio de janeiro
7	CUST163	Gabriela Almeida	gabriela.almeida@yahoo.com	03/13/2022	belo horizonte
8	CUST129	Gabriela Santos	gabriela.santos@yahoo.com	2023-03-20	Rio de Janeiro
9	CUST236	Joao Almeida	joao.almeida@gmail.com	06 Jan 2022	sao paulo
10	CUST169	Bruno Pereira	bruno.pereira@hotmail.com	2022-11-04	belo horizonte

Evidence 10

Sample order rows from RAW_ORDERS.



```
SELECT
    'RAW_ORDERS',
    COUNT(*)
FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS;

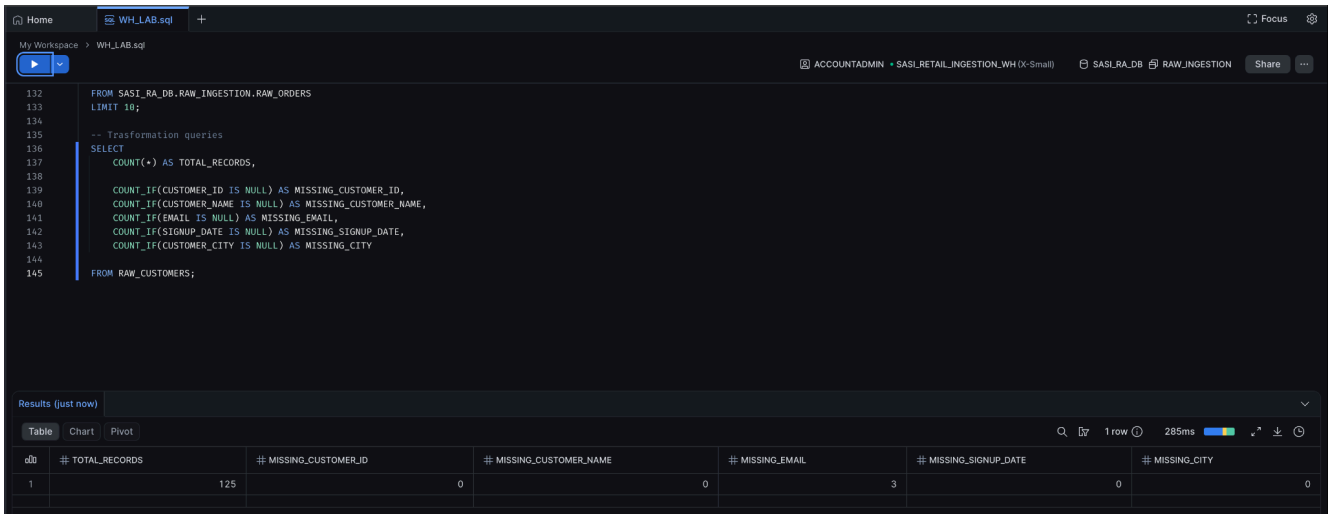
-- Inspect customer records
SELECT *
FROM SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS
LIMIT 10;

-- Inspect order records
SELECT *
FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS
LIMIT 10;
```

	ORDER_ID	CUSTOMER_ID	PURCHASE_DATE	DELIVERY_DATE	PRODUCT_CATEGORY	PRICE	QUANTITY	PAYMENT_TYPE	CUSTOMER_CITY	REVIEW_SCORE	REVIEW_COMMENT
	ORD10... 10.0% ORD10... 10.0% +8 more	CUST132 10.0% CUST167 10.0% +8 more	01/25/2024 10.0% 05/02/2024 10.0% +8 more	02/10/2024 10.0% 04/30/2024 10.0% +7 more	Electronics 20.0% Toys 20.0% +5 more	$6... 10.0% -- 10.0% +8 more	1 50.0% 4 50.0%	PayPal 30.0% Credit Card 20.0% +4 more	Belo Horizonte 20.0% Rio de Janeiro 20.0% +6 more	5.0 50.0% 3.0 20.0% +1 more	Not as expected 20.0%
1	ORD1175	CUST197	01/25/2024	2024-02-07	Electronics	295.63	1	PayPal	sao paulo	4.0	null
2	ORD1103	CUST234	28-01-2024	February 02, 2024	furniture	--	4	cash	Rio de Janeiro	null	null
3	ORD1134	CUST220	08 Feb 2024	02/10/2024	furniture	269.7	4	Debit Card	São Paulo	3.0	null
4	ORD1169	CUST132	26-09-2024	null	Toys	283.9	1	Credit Card	Belo Horizonte	5.0	null
5	ORD1104	CUST167	18-03-2024	05-04-2024	electronics	167.89	4	Debit Card	Rio de Janeiro	4.0	null
6	ORD1184	CUST267	June 23, 2024	2024-06-25	Toys	211.86	1	PayPal	belo horizonte	5.0	Not as expected
7	ORD1101	CUST279	11 Feb 2024	20 Feb 2024	Sports	$60.05	1	paypal	Curitiba	5.0	null
8	ORD1074	CUST271	05/02/2024	04/30/2024	Clothing	312.14	4	PayPal	Salvador	5.0	null
9	ORD1052	CUST228	2024-07-15	2024-07-17	Electronics	187.9	1	credit card	Belo Horizonte	3.0	null
10	ORD1131	CUST223	March 30, 2024	09 Apr 2024	Furniture	153.81	4	Credit Card	BrasÃlia	5.0	Not as expected

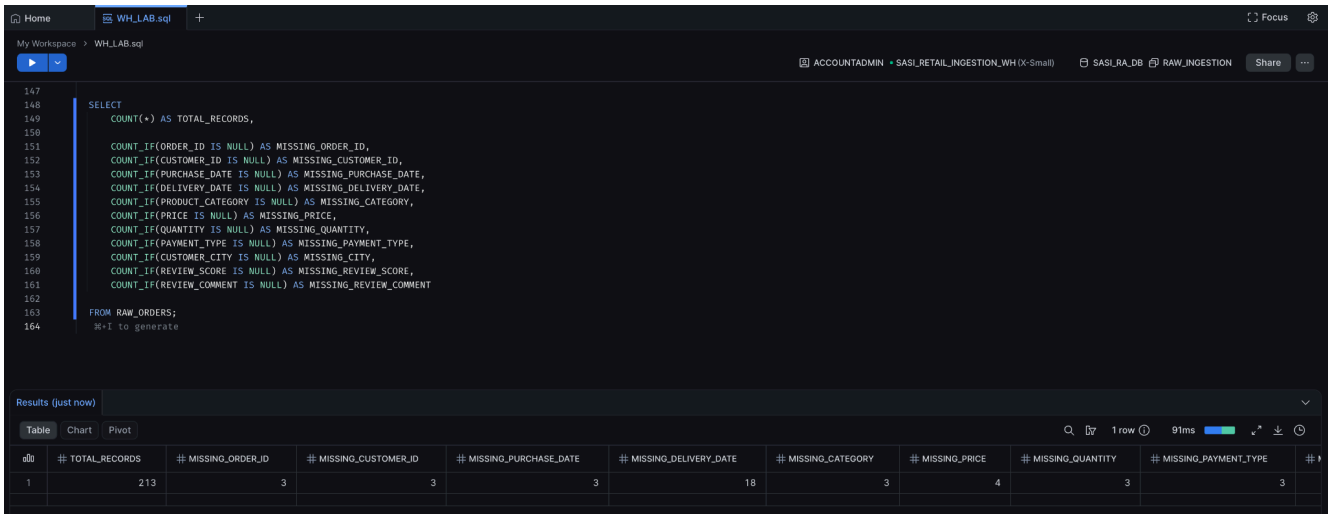
Evidence 11

Customer NULL profiling: three missing email values.



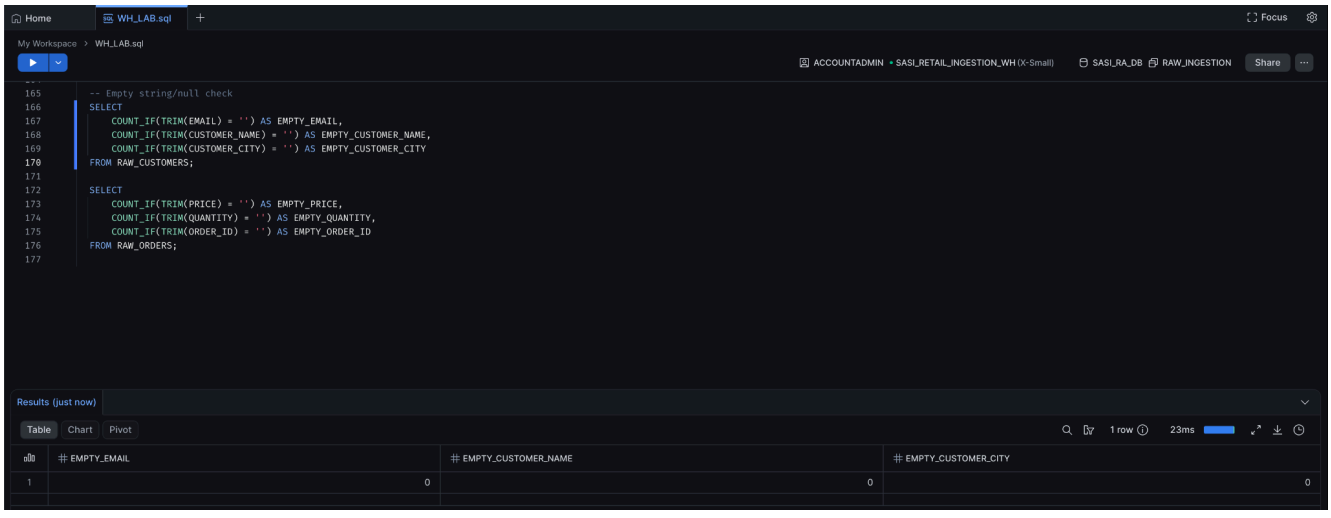
Evidence 12

Order NULL profiling: missing values across order columns.



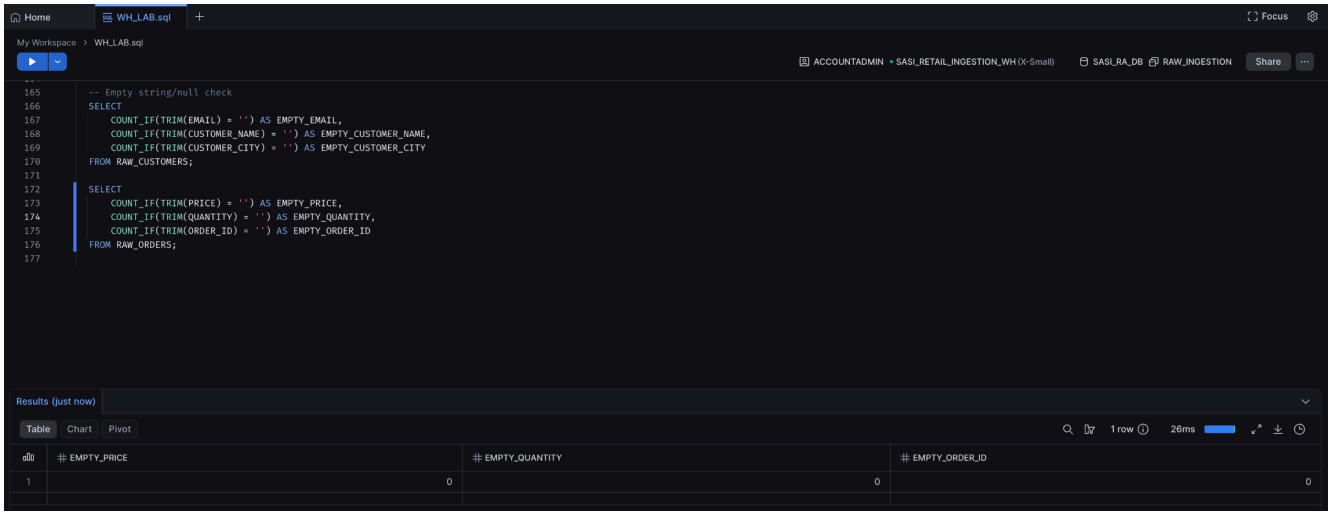
Evidence 13

Empty-string checks for selected customer columns.



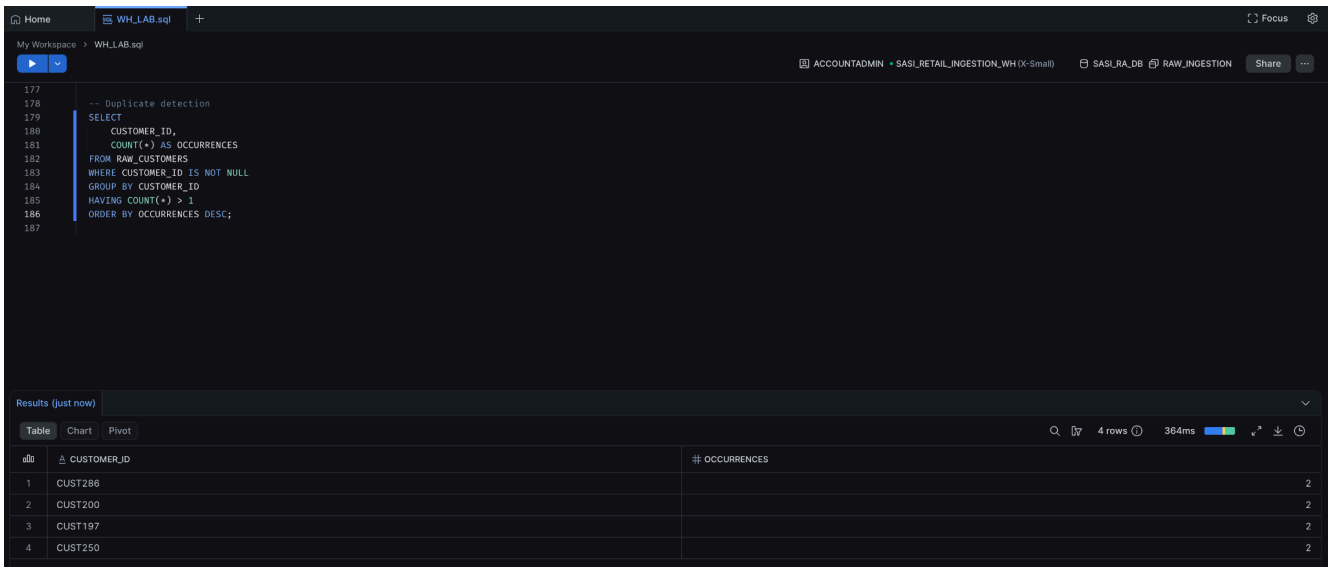
Evidence 14

Empty-string checks for selected order columns.



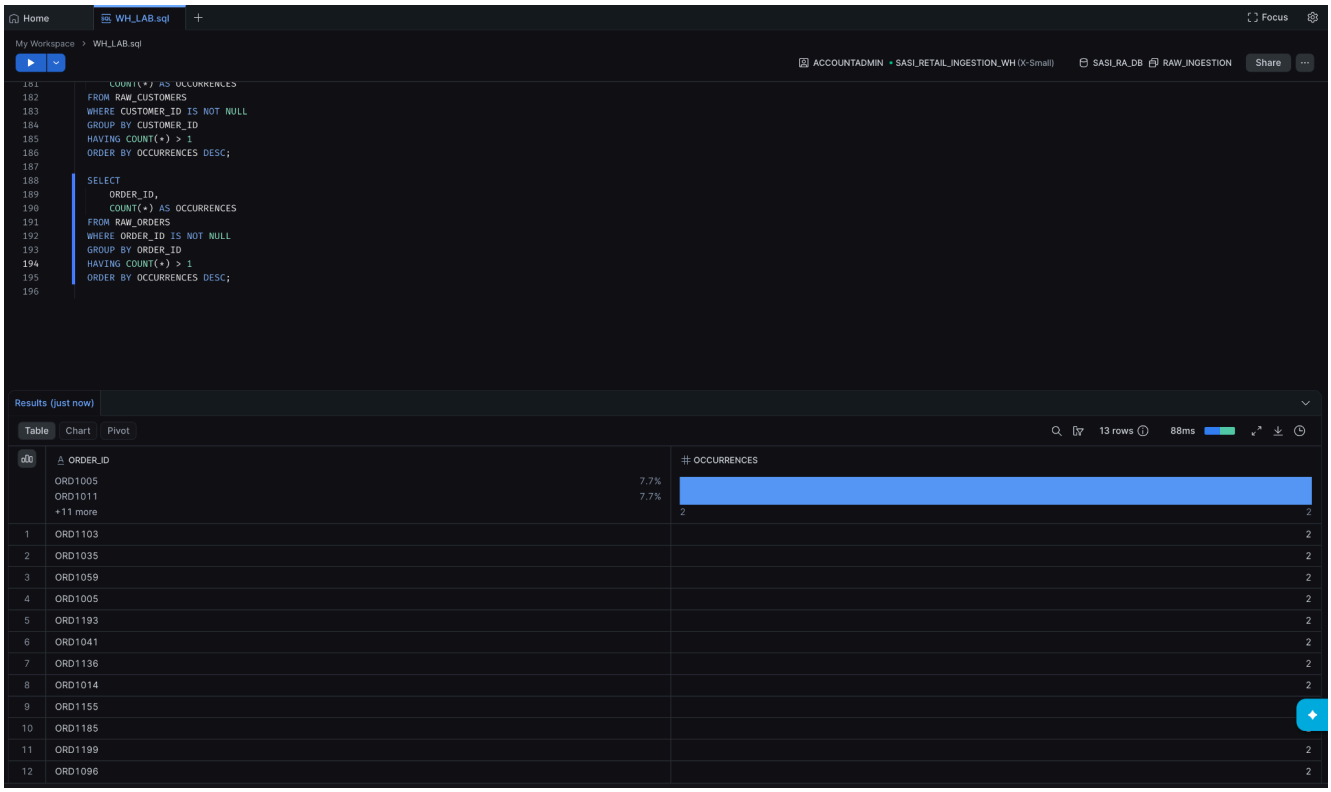
Evidence 15

Repeated customer identifier detection: four IDs shown twice.



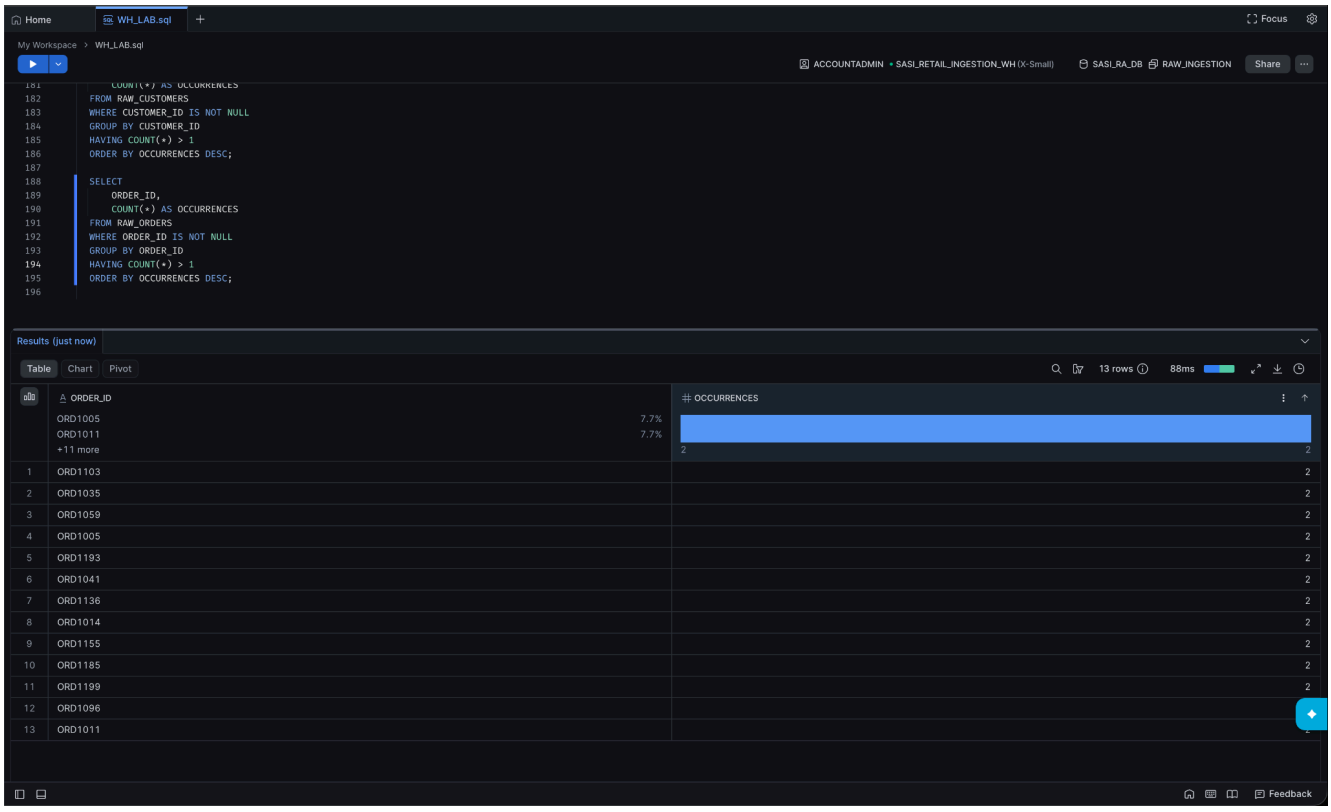
Evidence 16

Repeated order identifier detection: 13 rows shown in the result.



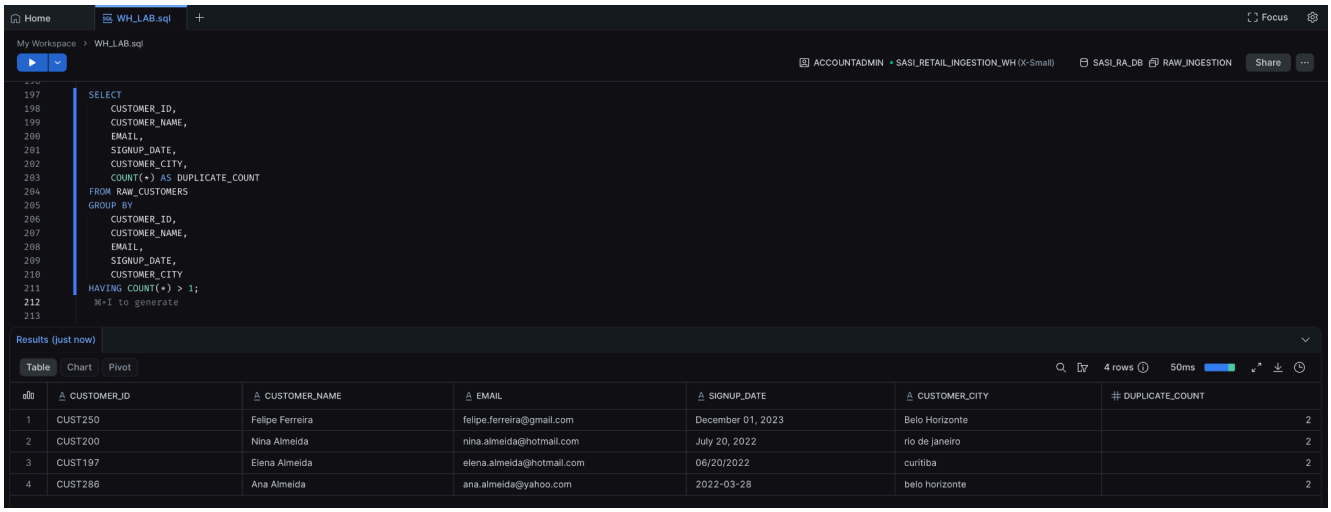
Evidence 17

Full-row duplicate check for customer records.



Evidence 18

Full-row duplicate check for order records.

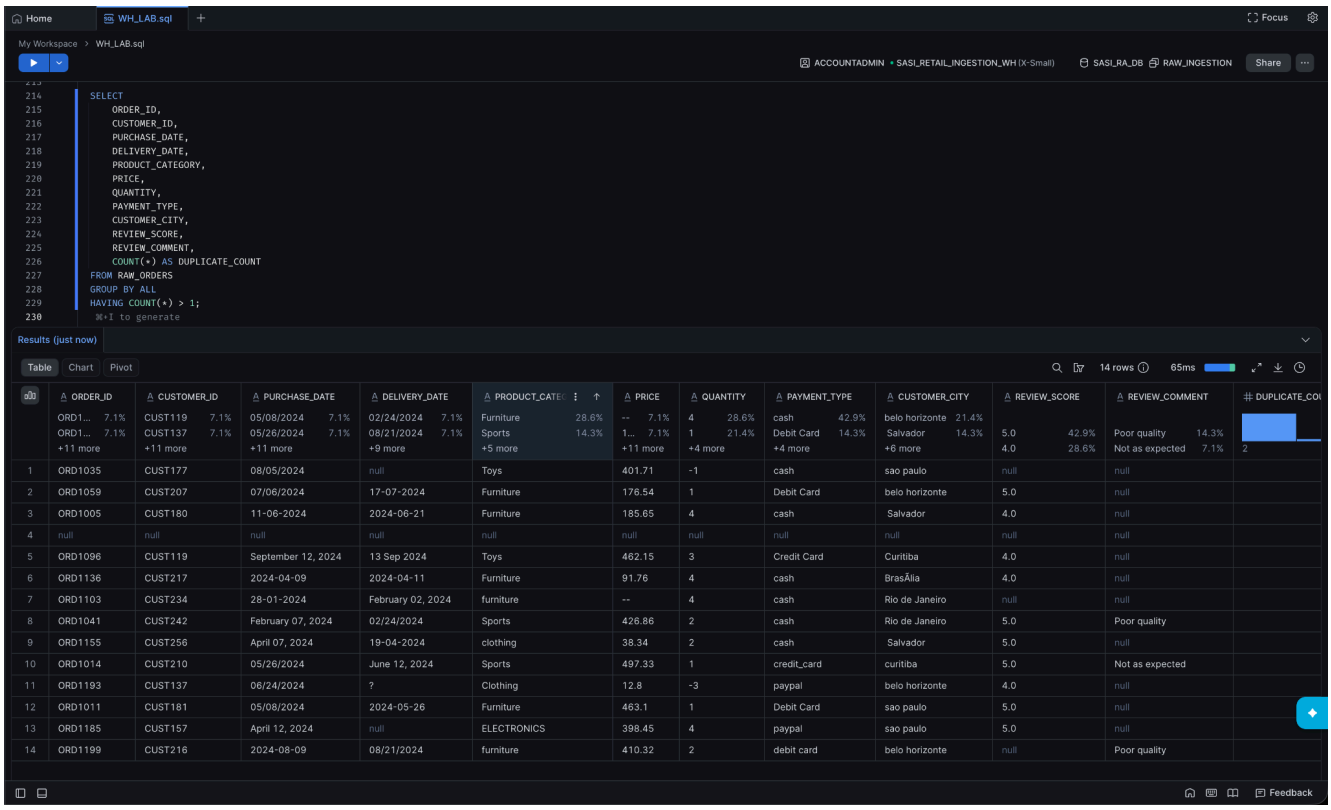


```
SELECT
    CUSTOMER_ID,
    CUSTOMER_NAME,
    EMAIL,
    SIGNUP_DATE,
    CUSTOMER_CITY,
    COUNT(*) AS DUPLICATE_COUNT
FROM RAW_CUSTOMERS
GROUP BY
    CUSTOMER_ID,
    CUSTOMER_NAME,
    EMAIL,
    SIGNUP_DATE,
    CUSTOMER_CITY
HAVING COUNT(*) > 1;
```

	CUSTOMER_ID	CUSTOMER_NAME	EMAIL	SIGNUP_DATE	CUSTOMER_CITY	DUPLICATE_COUNT
1	CUST250	Felipe Ferreira	felipe.ferreira@gmail.com	December 01, 2023	Belo Horizonte	2
2	CUST200	Nina Almeida	nina.almeida@hotmail.com	July 20, 2022	rio de janeiro	2
3	CUST197	Elena Almeida	elena.almeida@hotmail.com	06/20/2022	curitiba	2
4	CUST286	Ana Almeida	ana.almeida@yahoo.com	2022-03-28	belo horizonte	2

Evidence 19

Additional view of the full-row duplicate result set.



```
SELECT
    ORDER_ID,
    CUSTOMER_ID,
    PURCHASE_DATE,
    DELIVERY_DATE,
    PRODUCT_CATEGORY,
    PRICE,
    QUANTITY,
    PAYMENT_TYPE,
    CUSTOMER_CITY,
    REVIEW_SCORE,
    REVIEW_COMMENT,
    COUNT(*) AS DUPLICATE_COUNT
FROM RAW_ORDERS
GROUP BY ALL
HAVING COUNT(*) > 1;
```

	ORDER_ID	CUSTOMER_ID	PURCHASE_DATE	DELIVERY_DATE	PRODUCT_CATE	PRICE	QUANTITY	PAYMENT_TYPE	CUSTOMER_CITY	REVIEW_SCORE	REVIEW_COMMENT	DUPLICATE_CO
	ORD1... 7.1% ORD1... 7.1% +11 more	CUST119 7.1% CUST137 7.1% +11 more	05/08/2024 7.1% 05/26/2024 7.1% +11 more	02/24/2024 7.1% 08/21/2024 7.1% +9 more	Furniture 28.6% Sports 14.3% +5 more	-- 7.1% 1... 7.1% +11 more	4 28.6% 1 21.4% +4 more	cash 42.9% Debit Card 14.3% +4 more	belo horizonte 21.4% Salvador 14.3% +6 more	5.0 42.9% 4.0 28.6%	Poor quality 14.3% Not as expected 7.1%	2
1	ORD1035	CUST177	08/05/2024	null	Toys	401.71	-1	cash	sao paulo	null	null	
2	ORD1059	CUST207	07/06/2024	17-07-2024	Furniture	176.54	1	Debit Card	belo horizonte	5.0	null	
3	ORD1005	CUST180	11-06-2024	2024-06-21	Furniture	185.65	4	cash	Salvador	4.0	null	
4	null	null	null	null	null	null	null	null	null	null	null	
5	ORD1096	CUST119	September 12, 2024	13 Sep 2024	Toys	462.15	3	Credit Card	Curitiba	4.0	null	
6	ORD1136	CUST217	2024-04-09	2024-04-11	Furniture	91.76	4	cash	BrasÃlia	4.0	null	
7	ORD1103	CUST234	28-01-2024	February 02, 2024	furniture	--	4	cash	Rio de Janeiro	null	null	
8	ORD1041	CUST242	February 07, 2024	02/24/2024	Sports	426.86	2	cash	Rio de Janeiro	5.0	Poor quality	
9	ORD1155	CUST256	April 07, 2024	19-04-2024	clothing	38.34	2	cash	Salvador	5.0	null	
10	ORD1014	CUST210	05/26/2024	June 12, 2024	Sports	497.33	1	credit_card	curitiba	5.0	Not as expected	
11	ORD1193	CUST137	06/24/2024	?	Clothing	12.8	-3	paypal	belo horizonte	4.0	null	
12	ORD1011	CUST181	05/08/2024	2024-05-26	Furniture	463.1	1	Debit Card	sao paulo	5.0	null	
13	ORD1185	CUST157	April 12, 2024	null	ELECTRONICS	398.45	4	paypal	sao paulo	5.0	null	
14	ORD1199	CUST216	2024-08-09	08/21/2024	furniture	410.32	2	debit card	belo horizonte	null	Poor quality	

REVISION 2 — DATA PROFILING, VALIDATION & STAGING PREPARATION

Additional evidence supplied on 05 November 2026

14. Extended profiling and normalization

The additional worksheet work moves beyond confirming that the CSV files loaded successfully. It profiles the raw values, tests whether source fields can be converted safely, checks relationships between the customer and order datasets, and identifies records that need remediation before a curated analytics layer is built. The source tables remain in the RAW_INGESTION schema; the checks use TRY_TO_DATE, TRY_TO_DECIMAL and TRY_TO_NUMBER so malformed values return NULL rather than aborting the entire query.

14.1 Signup-date profiling and parsing

A frequency query groups RAW_CUSTOMERS by the original SIGNUP_DATE string. The result contains 120 distinct non-NULL date strings/values in the grouped output, demonstrating that the source is not stored in one uniform textual representation. A second query creates PARSED_SIGNUP_DATE by trying several explicit formats in sequence.

```
SELECT
    SIGNUP_DATE,
    COALESCE(
        TRY_TO_DATE(TRIM(SIGNUP_DATE), 'YYYY-MM-DD'),
        TRY_TO_DATE(TRIM(SIGNUP_DATE), 'MM/DD/YYYY'),
        TRY_TO_DATE(TRIM(SIGNUP_DATE), 'DD-MM-YYYY'),
        TRY_TO_DATE(TRIM(SIGNUP_DATE), 'DD MON YYYY'),
        TRY_TO_DATE(TRIM(SIGNUP_DATE), 'MMMM DD, YYYY')
    ) AS PARSED_SIGNUP_DATE
FROM RAW_CUSTOMERS;
```

The captured examples normalize successfully, including values such as 06/01/2022, 28-01-2022, 16 Apr 2023 and March 04, 2023. The parsed result spans 2022-01-01 through 2023-12-01 in the displayed histogram. The format list is deliberately explicit: ambiguous numeric dates such as 01/02/2022 require a documented source convention (MM/DD/YYYY versus DD/MM/YYYY) because both interpretations may be valid dates.

14.2 Purchase and delivery date normalization

RAW_ORDERS contains mixed purchase-date and delivery-date strings. The profiling query groups by the raw pair to expose inconsistent representations. The normalization query then parses each field independently using the same ordered format list. A validation query filtering for non-NULL purchase dates that cannot be parsed returned zero rows in the captured run.

```
COALESCE(
    TRY_TO_DATE(TRIM(PURCHASE_DATE), 'YYYY-MM-DD'),
    TRY_TO_DATE(TRIM(PURCHASE_DATE), 'MM/DD/YYYY'),
    TRY_TO_DATE(TRIM(PURCHASE_DATE), 'DD-MM-YYYY'),
    TRY_TO_DATE(TRIM(PURCHASE_DATE), 'DD MON YYYY'),
    TRY_TO_DATE(TRIM(PURCHASE_DATE), 'MMMM DD, YYYY')
) AS PARSED_PURCHASE_DATE
```

The corresponding expression is applied to DELIVERY_DATE. In the final date-quality summary, the 213 order rows comprise 18 missing delivery dates, 6 non-empty delivery-date values that fail the parser, and 189 successfully parsed delivery dates. These categories reconcile to the full order-row count (18 + 6 + 189 = 213).

14.3 Price conversion and business-rule validation

Price strings are cleaned by removing characters other than digits, decimal points and minus signs, then converted to DECIMAL(12,2) with TRY_TO_DECIMAL. The invalid-conversion filter returned zero rows in the captured run, indicating

that the non-NULL source values could be converted after the cleanup expression. This does not mean all prices are valid business values: a separate <= 0 check returned six rows, all with negative prices.

```
TRY_TO_DECIMAL(
    REGEXP_REPLACE(TRIM(PRICE), '[^0-9.-]', ''),
    12,
    2
) AS PARSED_PRICE
```

The six negative-price examples include -222.86, -471.22, -124.47, -119.56, -394.06 and -40.72. The profiling histogram also shows a wide range, approximately -471.22 to 9,692.07. Negative values should be investigated as possible refunds, reversals, sign errors or bad source records rather than silently converted to positive values. The regular expression is a parsing aid, not a complete currency-validation policy; malformed strings containing multiple decimal points should be rejected by a stricter validation rule.

14.4 Quantity and review-score validation

Quantity is converted using TRY_TO_NUMBER(TRIM(QUANTITY)). The captured profiling output includes negative values and non-numeric placeholders. The combined validation query for non-NULL quantities that fail numeric conversion or are less than/equal to zero returned 15 rows. These records require business review; negative quantities may represent returns only if the source system explicitly uses that convention.

```
SELECT *
FROM RAW_ORDERS
WHERE QUANTITY IS NOT NULL
  AND (
      TRY_TO_NUMBER(TRIM(QUANTITY)) IS NULL
      OR TRY_TO_NUMBER(TRIM(QUANTITY)) <= 0
  );
```

Review scores were tested with TRY_TO_DECIMAL and a 1–5 range condition. The invalid-value query returned zero rows, but the consolidated quality report separately records 18 missing review scores. A zero-invalid result therefore must not be interpreted as zero missing values: missingness and invalidity are separate quality dimensions.

15. Referential integrity and relationship checks

15.1 Orders referencing missing customers

A LEFT JOIN from RAW_ORDERS to RAW_CUSTOMERS, using trimmed customer IDs, identifies order-side customer IDs that have no matching customer record. Six unmatched customer IDs were returned, with affected-order counts of 2, 2, 2, 1, 1 and 1 respectively (nine affected order rows in total). This is a referential-integrity exception and should be resolved or explicitly classified before building a customer dimension relationship.

```
SELECT
    O.CUSTOMER_ID,
    COUNT(*) AS AFFECTED_ORDERS
FROM RAW_ORDERS O
LEFT JOIN RAW_CUSTOMERS C
  ON TRIM(O.CUSTOMER_ID) = TRIM(C.CUSTOMER_ID)
WHERE O.CUSTOMER_ID IS NOT NULL
  AND C.CUSTOMER_ID IS NULL
GROUP BY O.CUSTOMER_ID
ORDER BY AFFECTED_ORDERS DESC;
```

15.2 Customers with no orders

The reverse LEFT JOIN identifies customer records without a matching order. One customer was returned: CUST248 (Bruno Pereira). This is not automatically an error; it may represent a newly registered customer who has not purchased yet. It is useful for customer lifecycle and conversion analysis, and should not be removed merely because no order exists.

16. Temporal consistency checks

After parsing purchase and delivery dates, the lab checks for DELIVERY_DT < PURCHASE_DT. Ten order rows were returned. The examples include ORD1074, ORD1039, ORD1160, ORD1068, ORD1017, ORD1188, ORD1118 and ORD1080. These rows have delivery dates earlier than their purchase dates and should be quarantined or investigated. The check is meaningful only after both fields have been normalized to DATE values; comparing raw strings would produce unreliable results.

17. Consolidated data-quality report

The UNION ALL quality-check query produces a compact issue register across the two raw datasets. The captured output is summarized below.

Dataset	Check	Issue count	Captured status
CUSTOMERS	MISSING_CUSTOMER_ID	0	PASS
CUSTOMERS	MISSING_EMAIL	3	REQUIRES ATTENTION
ORDERS	INVALID_PRICE	10	REQUIRES ATTENTION
ORDERS	INVALID_QUANTITY	15	REQUIRES ATTENTION
ORDERS	MISSING_DELIVERY_DATE	18	REQUIRES ATTENTION
ORDERS	MISSING_ORDER_ID	3	REQUIRES ATTENTION
ORDERS	MISSING_REVIEW_SCORE	18	REQUIRES ATTENTION

Important interpretation: the INVALID_PRICE count in this report is a separately defined quality rule and should not be conflated with the six negative prices found by the explicit <= 0 test. The report's INVALID_PRICE rule may include missing/placeholder values (for example '--', NULL, or values that cannot be parsed) depending on its exact expression. Keep the rule definition beside the metric in production so that the count is reproducible.

18. Duplicate-record profiling

18.1 Customer duplicate identifiers

Grouping RAW_CUSTOMERS by CUSTOMER_ID and comparing COUNT(*) with a COUNT(DISTINCT HASH(...)) over CUSTOMER_NAME, EMAIL, SIGNUP_DATE and CUSTOMER_CITY returned four customer IDs: CUST250, CUST286, CUST200 and CUST197. Each has two rows but only one distinct attribute-version hash. The evidence therefore points to exact duplicate customer records for these selected attributes, not conflicting customer versions. The chosen hash columns should be expanded if additional business-significant customer attributes are added to the source.

18.2 Duplicate order identifiers

A similar check grouped orders by ORDER_ID and hashed the complete set of displayed order attributes (customer, dates, category, price, quantity, payment type, city and review fields). Thirteen order IDs were returned with two rows and one distinct version each, indicating repeated identical records across the selected fields. Before deduplication, confirm whether the source file can legitimately contain multiple line items per order ID. If ORDER_ID identifies an order header rather than a unique line item, the correct grain may require an order-line identifier.

19. Staging schema preparation

The worksheet creates a separate STAGING schema inside SASI_RA_DB and switches the session to it. SHOW SCHEMAS IN DATABASE SASI_RA_DB confirms the database contains INFORMATION_SCHEMA, PUBLIC, RAW_INGESTION and STAGING. This is evidence of schema creation and session selection only; the screenshots do not yet demonstrate creation or population of curated staging tables.

```
CREATE SCHEMA IF NOT EXISTS SASI_RA_DB.STAGING;
USE SCHEMA SASI_RA_DB.STAGING;
SHOW SCHEMAS IN DATABASE SASI_RA_DB;
```

20. Findings, risks and recommended next steps

- Ingestion remains validated at 125 customer rows and 213 order rows; the additional work is profiling and schema preparation, not a replacement of the raw landing layer.
- Date parsing is largely successful for the captured purchase-date values, but mixed numeric formats require an explicit locale convention to prevent silent month/day swaps.
- Delivery dates have 18 missing values and 6 unparseable non-empty values; ten parsed records also violate the delivery-after-purchase rule.
- The price and quantity checks expose negative or otherwise questionable business values. These should be retained in raw data and handled through explicit exception rules.
- Six order-side customer IDs do not match the customer table; one customer has no orders. These are different relationship conditions and should be reported separately.
- Duplicate checks identify repeated customer and order records. Deduplication must respect the true grain and source-system semantics.
- The STAGING schema now exists, but the next evidence should show staging-table DDL, transformation logic, rejected-row handling and reconciliation to the raw row counts.

Suggested next implementation sequence

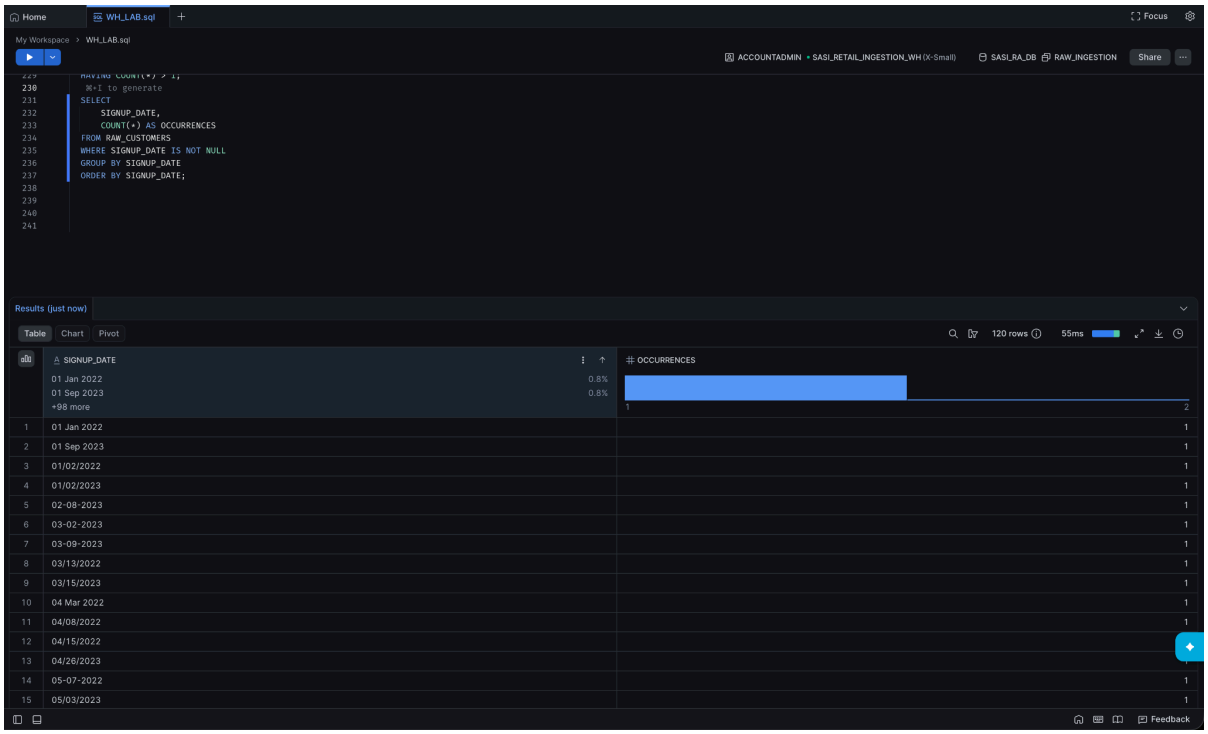
1. Define canonical column types and explicit date-format/locale rules.
2. Create typed staging tables with source-row identifiers and load timestamps.
3. Separate valid records from rejected records while preserving the raw source values and rejection reasons.
4. Implement deterministic deduplication using the confirmed business key and source ordering.
5. Resolve or classify orphan customer references and invalid temporal sequences.
6. Reconcile raw, accepted and rejected row counts for every load batch.
7. Build curated customer and order models only after the quality rules and table grain are agreed.

21. Revision 2 evidence register

The following 20 additional screenshots are embedded in the supplied order. Captions describe the query or result visible in each capture. Where the screenshot shows a query result rather than a complete SQL definition, the narrative records the observed outcome without assuming unseen logic.

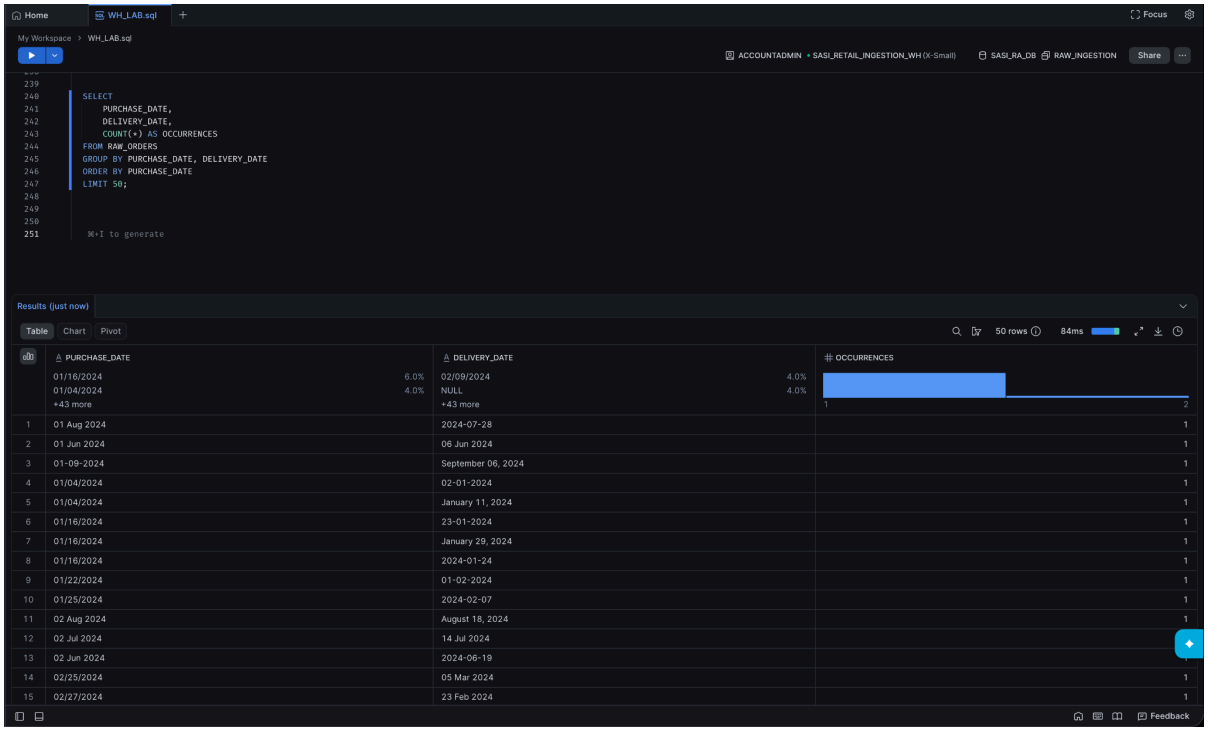
Evidence 20

Raw signup-date frequency profile in RAW_CUSTOMERS.



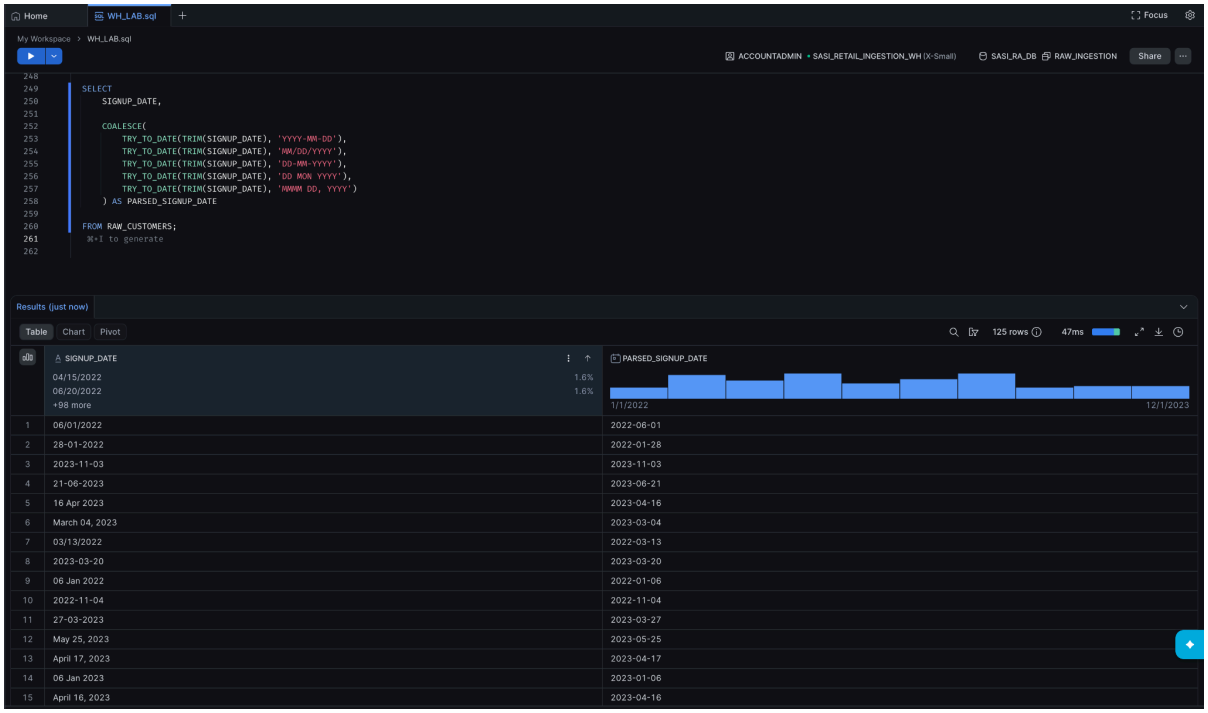
Evidence 21

Frequency profile of raw purchase-date and delivery-date pairs in RAW_ORDERS.



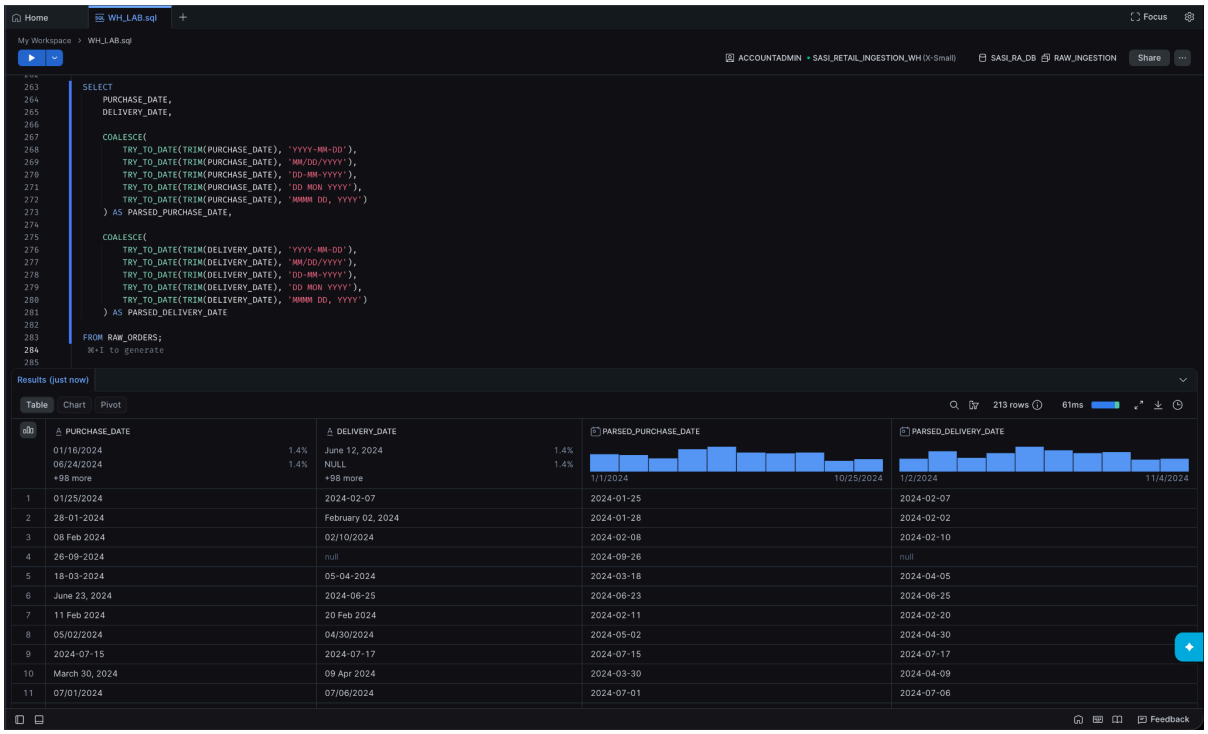
Evidence 22

Signup-date parsing with multiple TRY_TO_DATE formats.



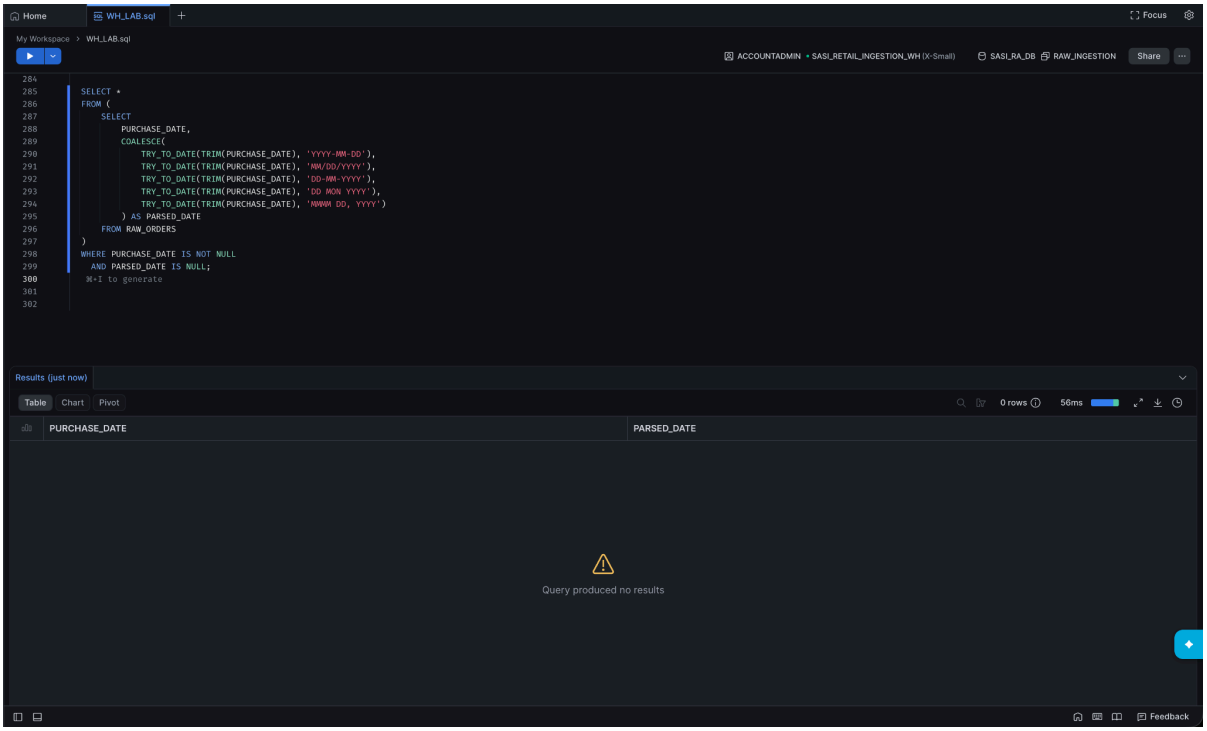
Evidence 23

Purchase-date and delivery-date normalization into DATE values.



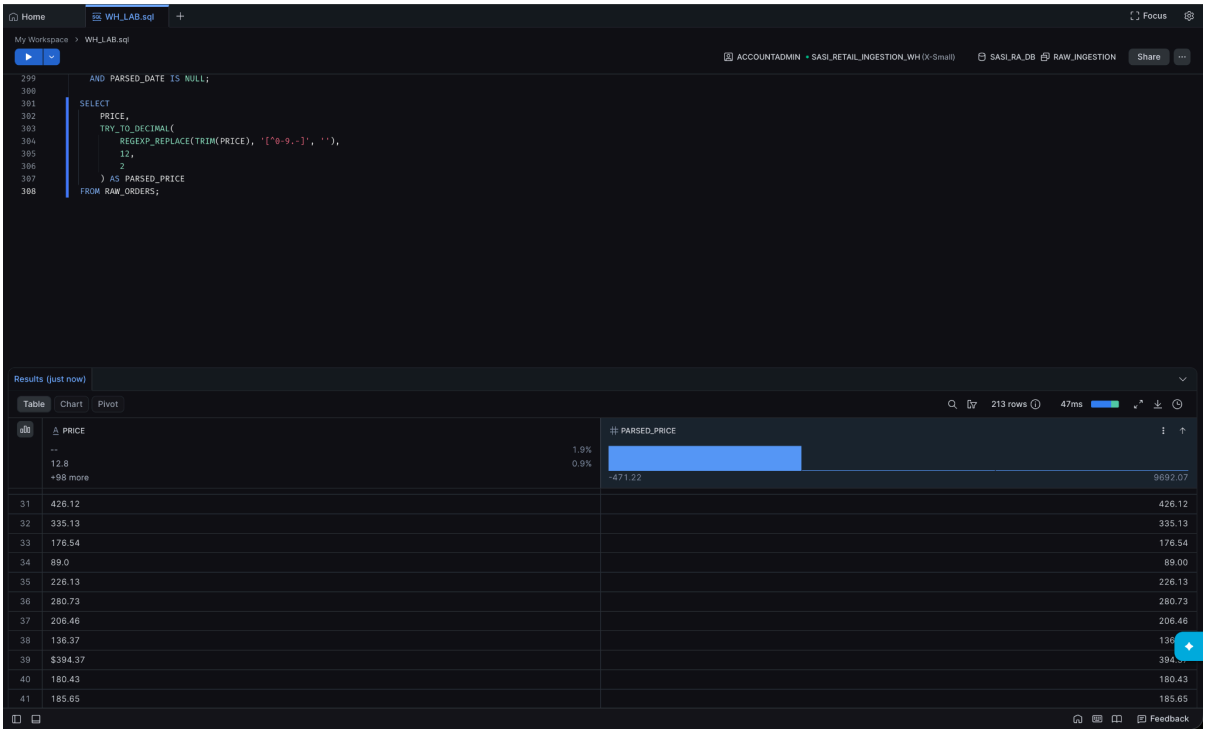
Evidence 24

Validation query for non-NULL purchase dates that could not be parsed; no rows returned.



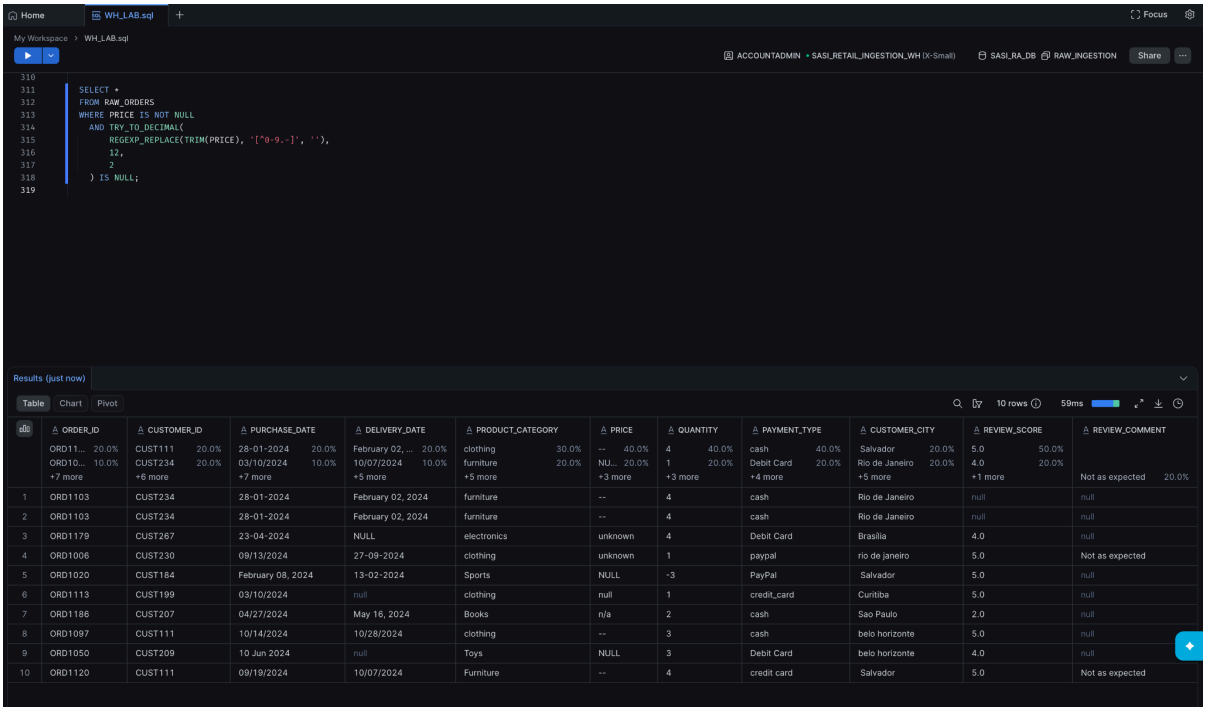
Evidence 25

Price cleanup and TRY_TO_DECIMAL conversion output.



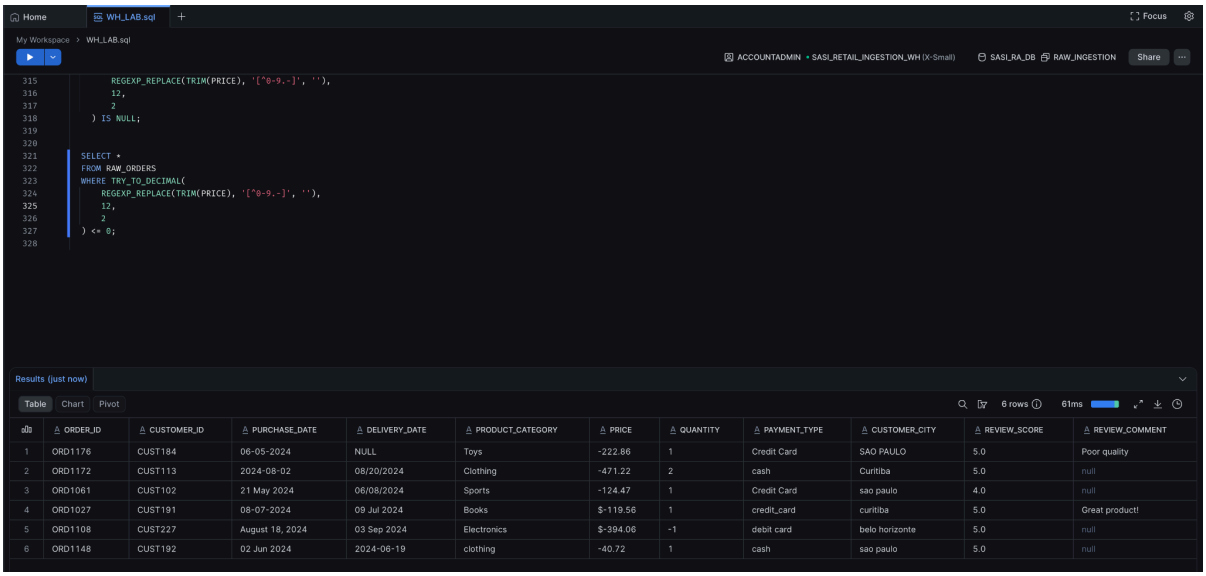
Evidence 26

Validation query for non-NULL prices that fail numeric conversion; no rows returned.



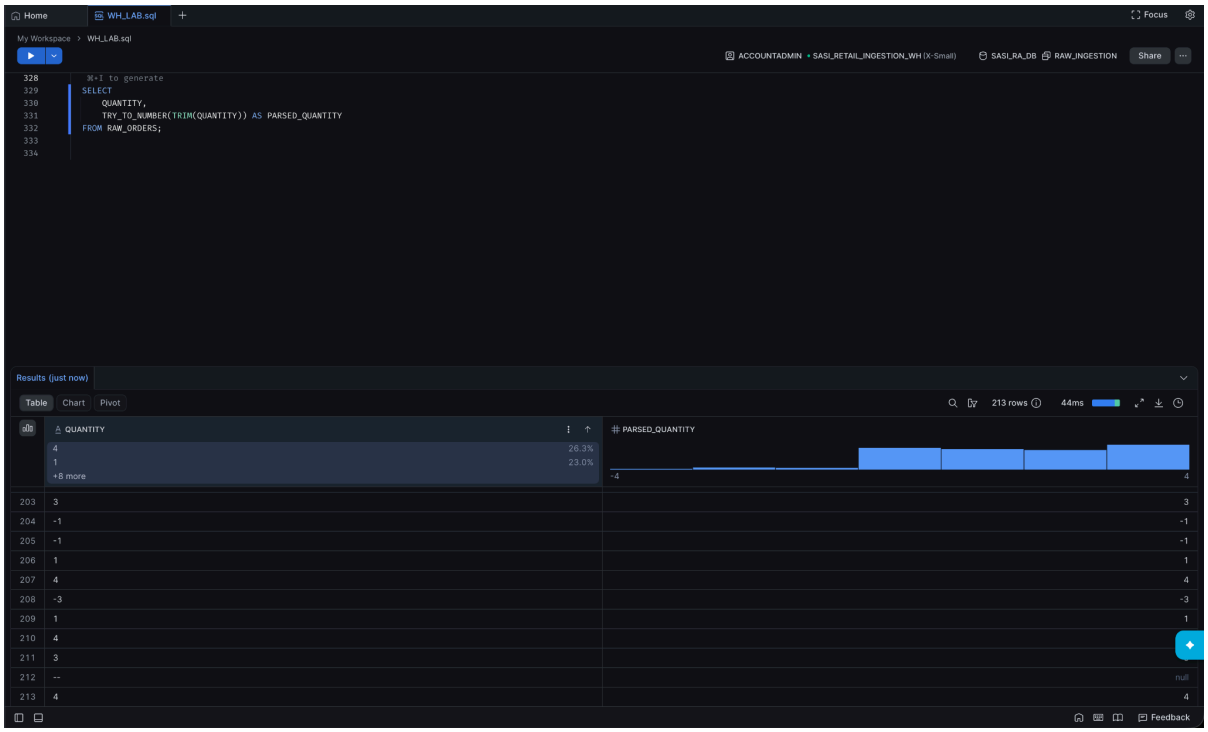
Evidence 27

Business-rule check for prices less than or equal to zero; six negative-price rows returned.



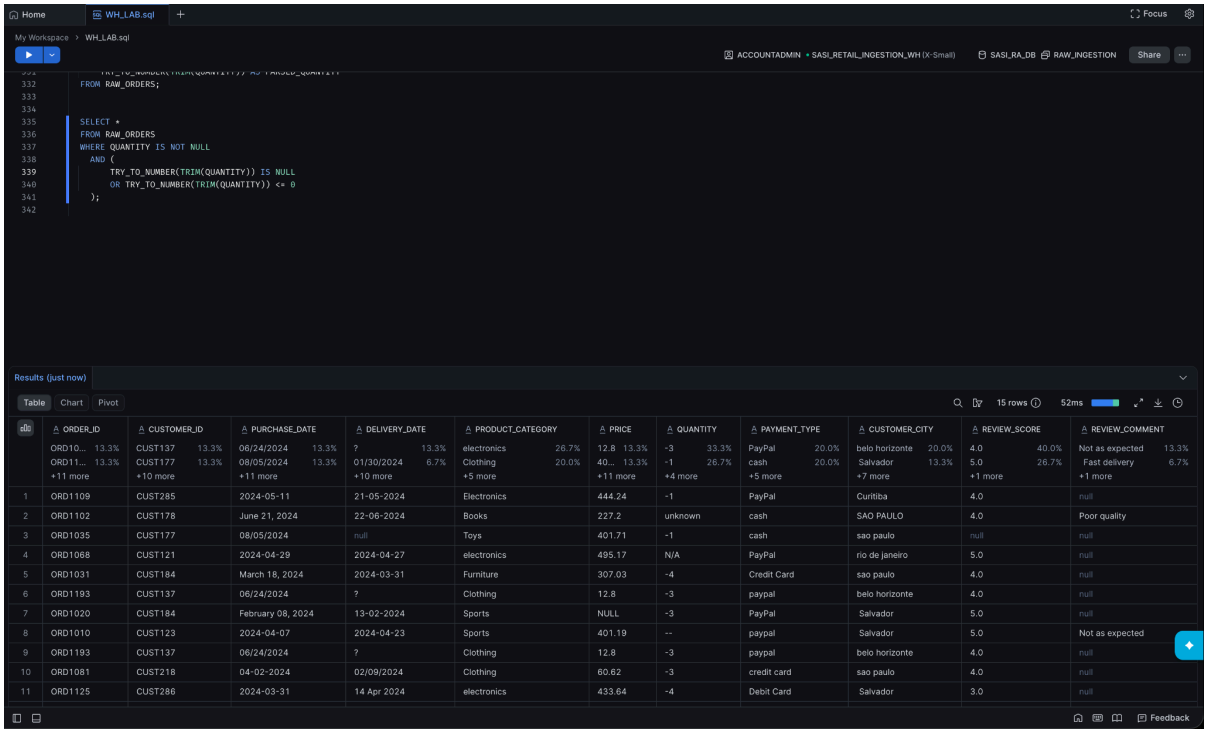
Evidence 28

Quantity conversion using TRY_TO_NUMBER and parsed quantity distribution.



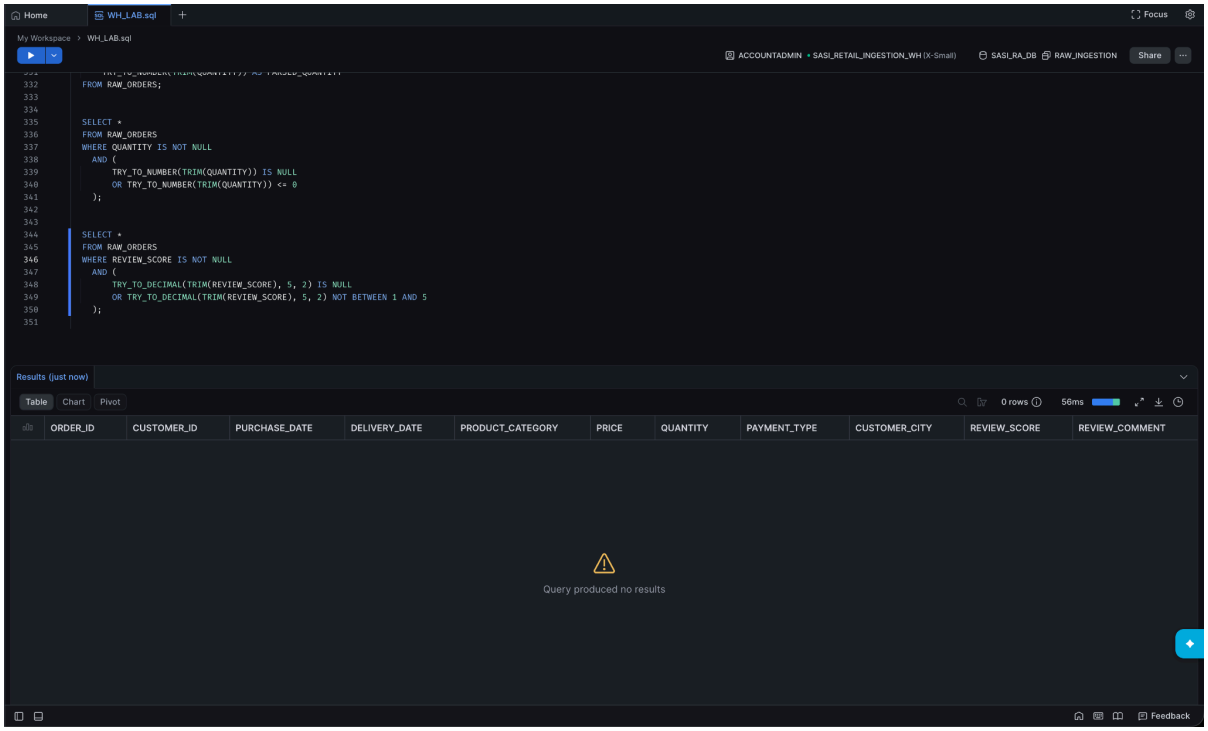
Evidence 29

Validation query for non-numeric or non-positive quantities; 15 rows returned.



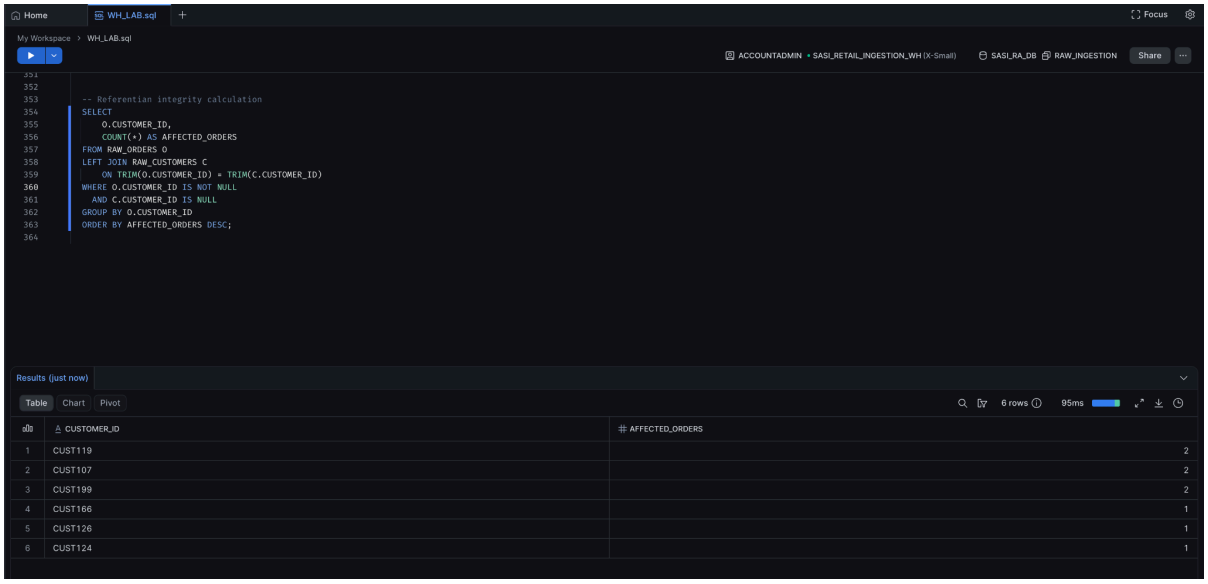
Evidence 30

Review-score conversion and 1–5 range validation; no invalid rows returned.



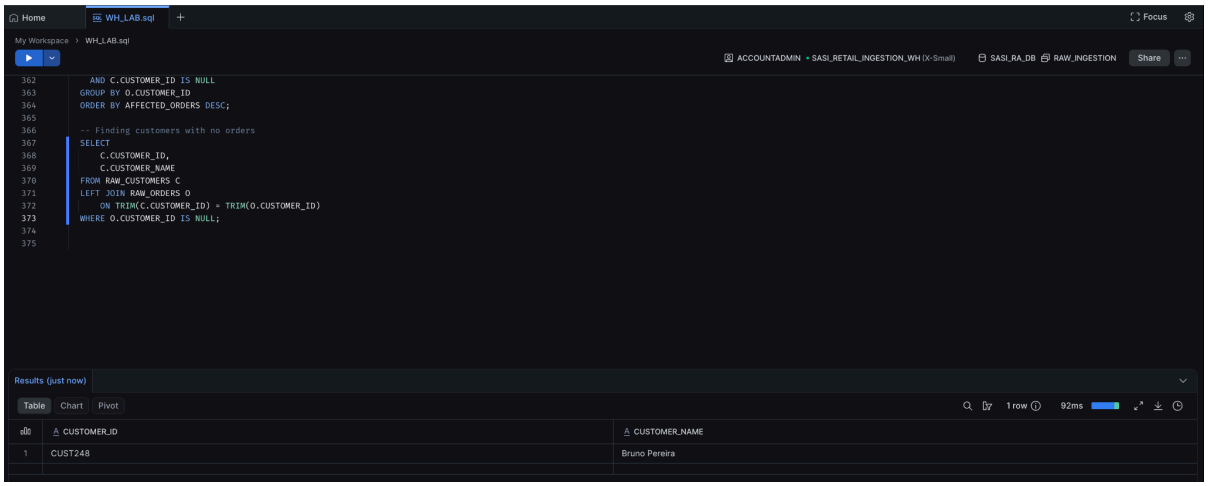
Evidence 31

Referential-integrity check: order customer IDs with no matching customer record.



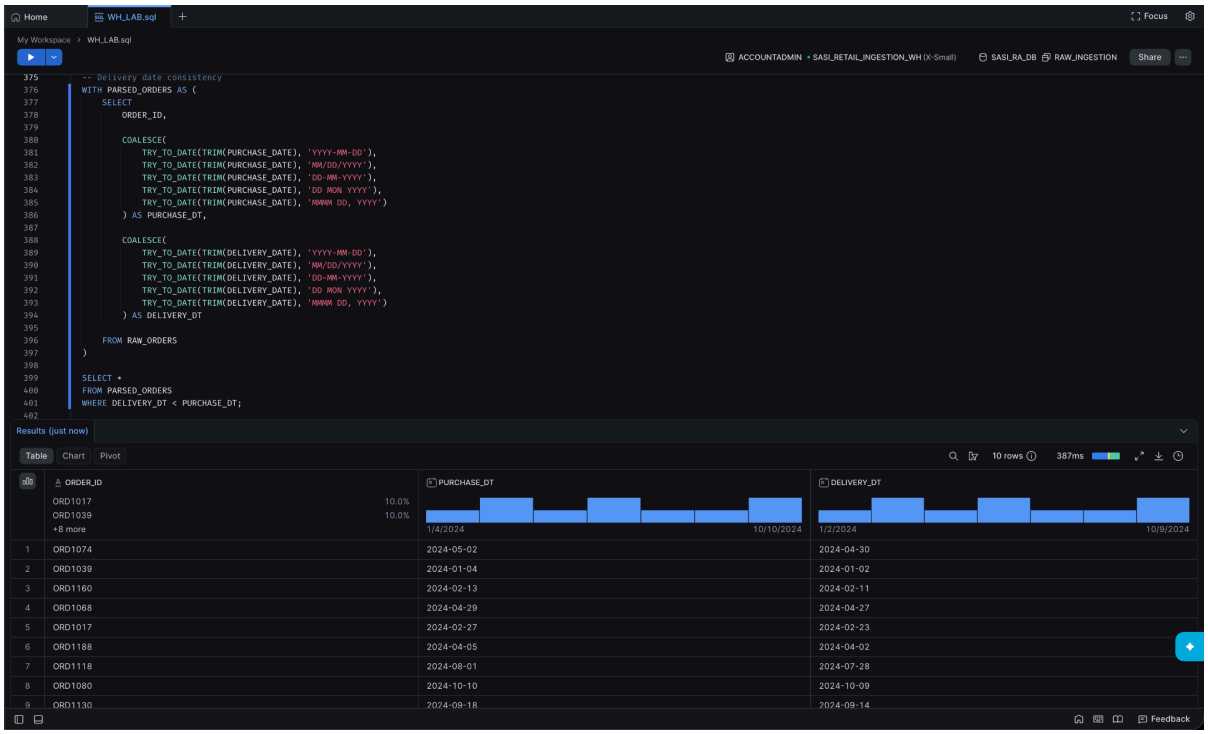
Evidence 32

Reverse relationship check: customers with no matching orders; CUST248 is returned.



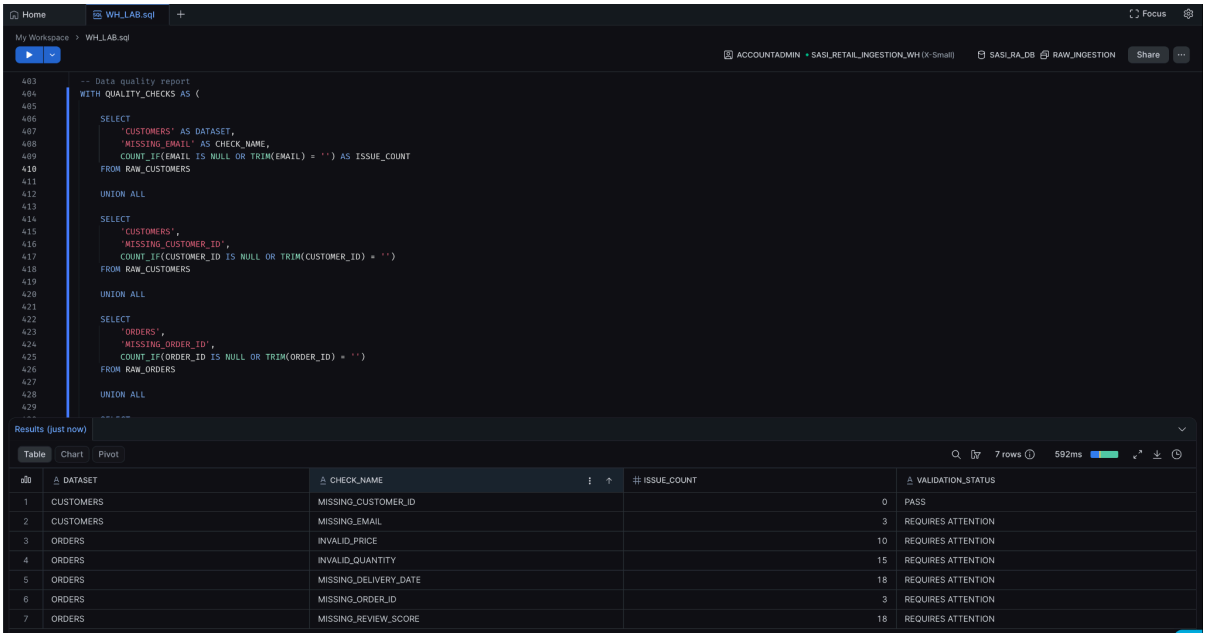
Evidence 33

Temporal consistency check for delivery dates earlier than purchase dates; ten rows returned.



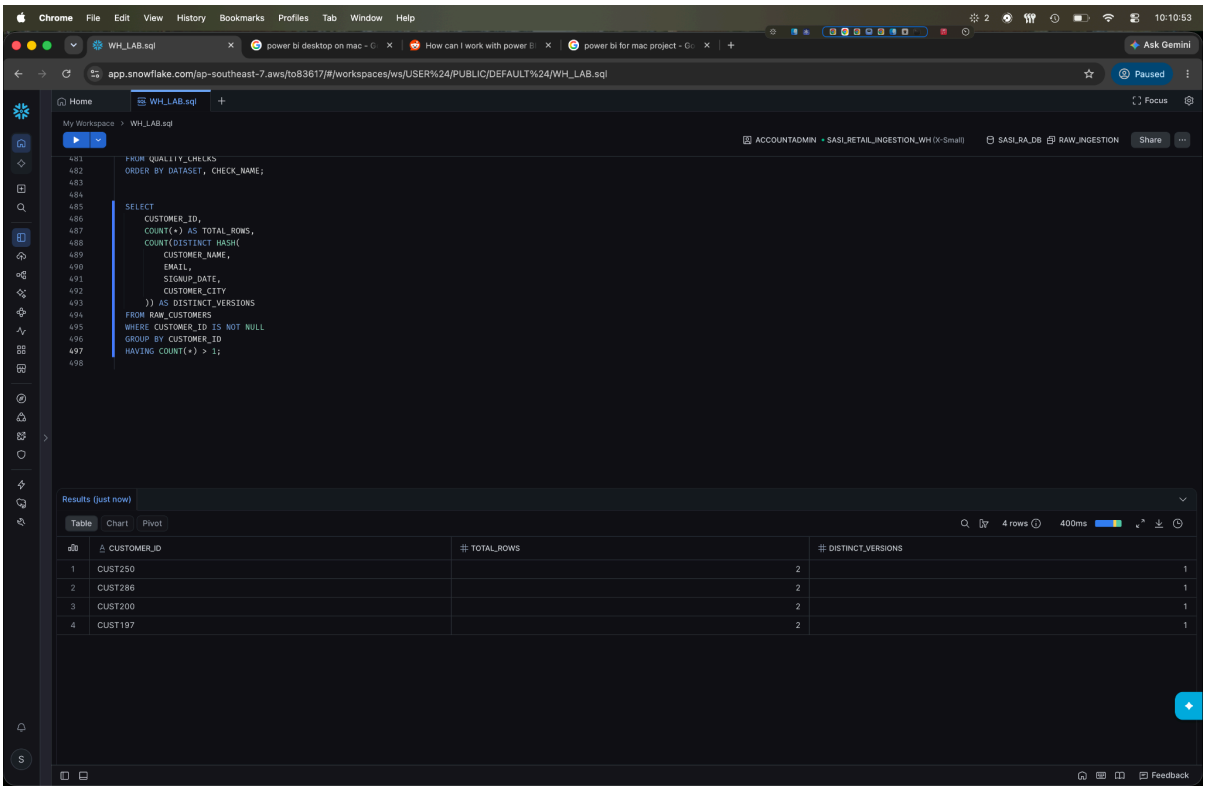
Evidence 34

Consolidated data-quality report across customer and order checks.



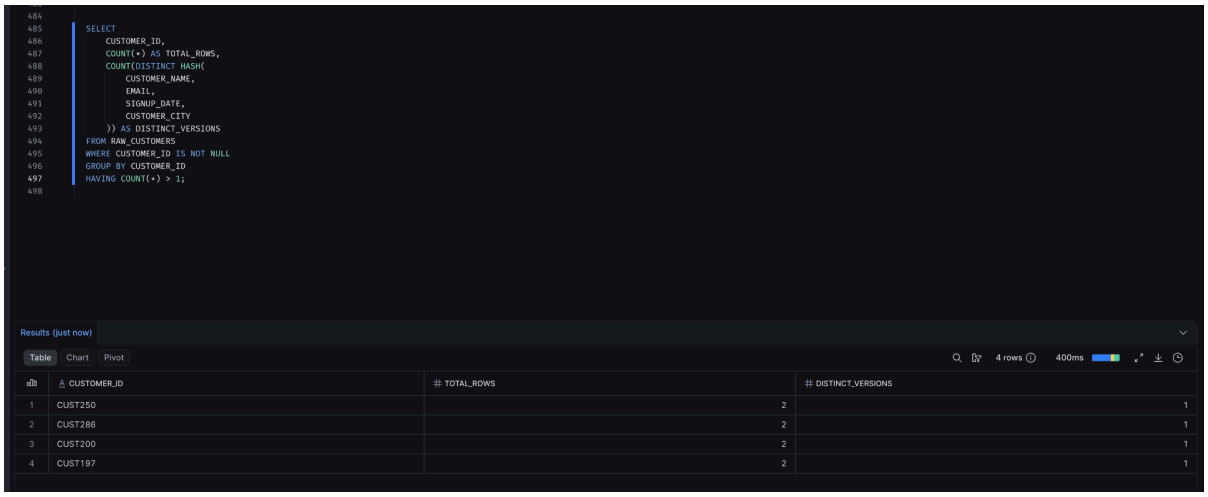
Evidence 35

Duplicate customer-ID profiling using row counts and distinct attribute hashes.



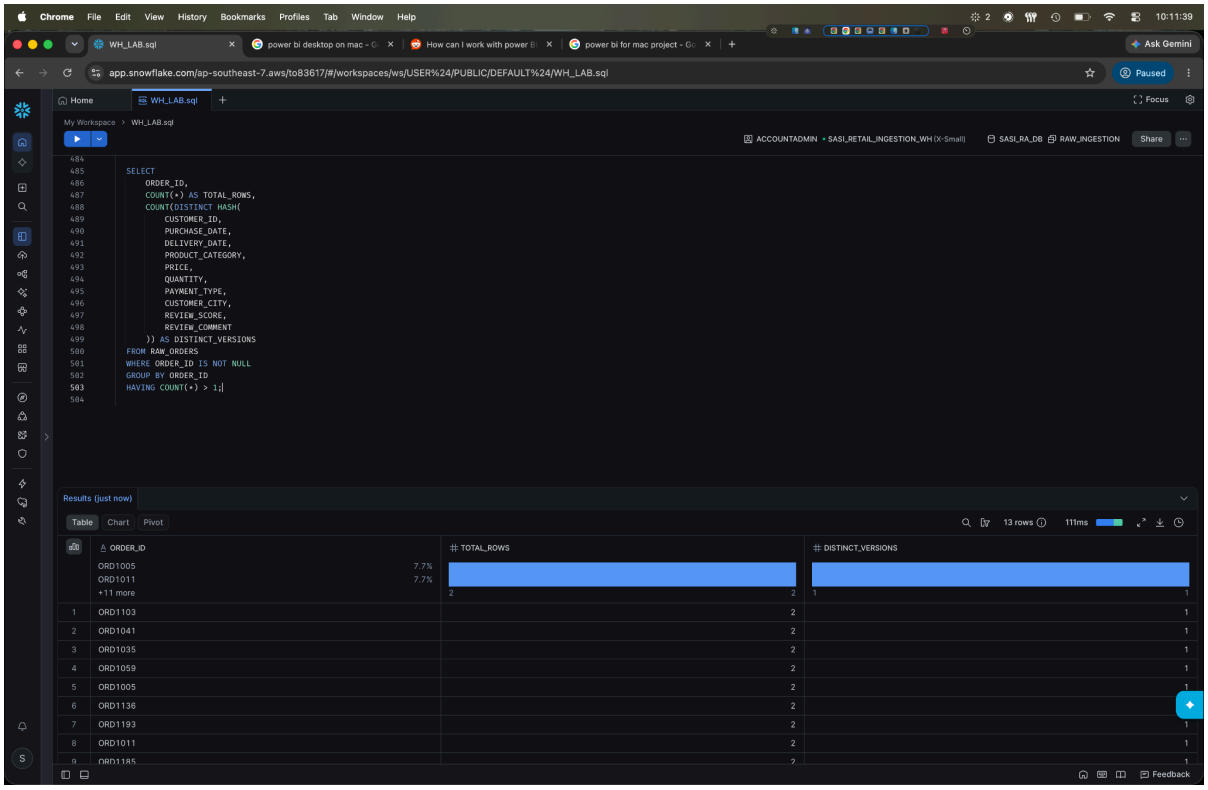
Evidence 36

Duplicate order-ID profiling using row counts and hashes of order attributes.



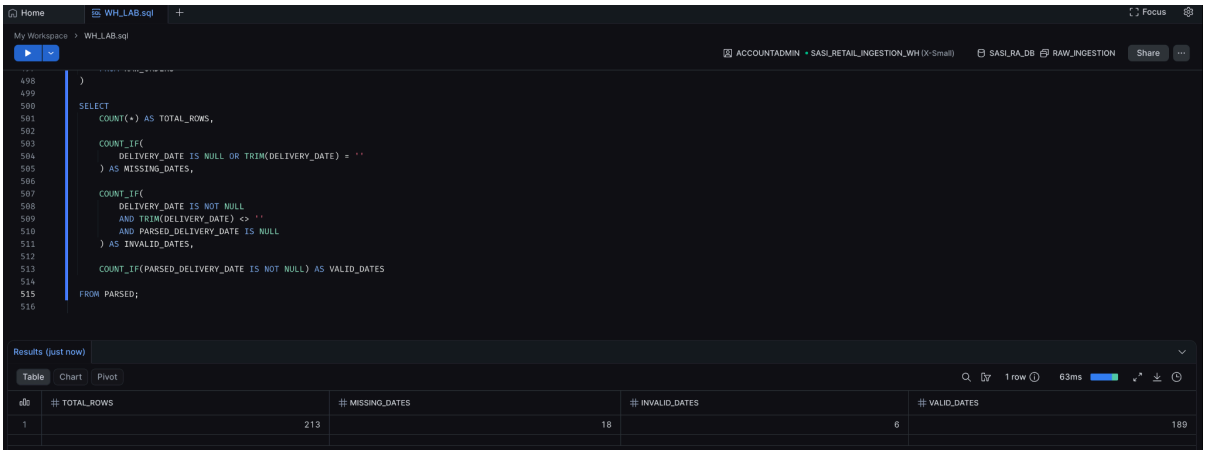
Evidence 37

Delivery-date completeness summary: 213 total, 18 missing, 6 invalid and 189 valid dates.



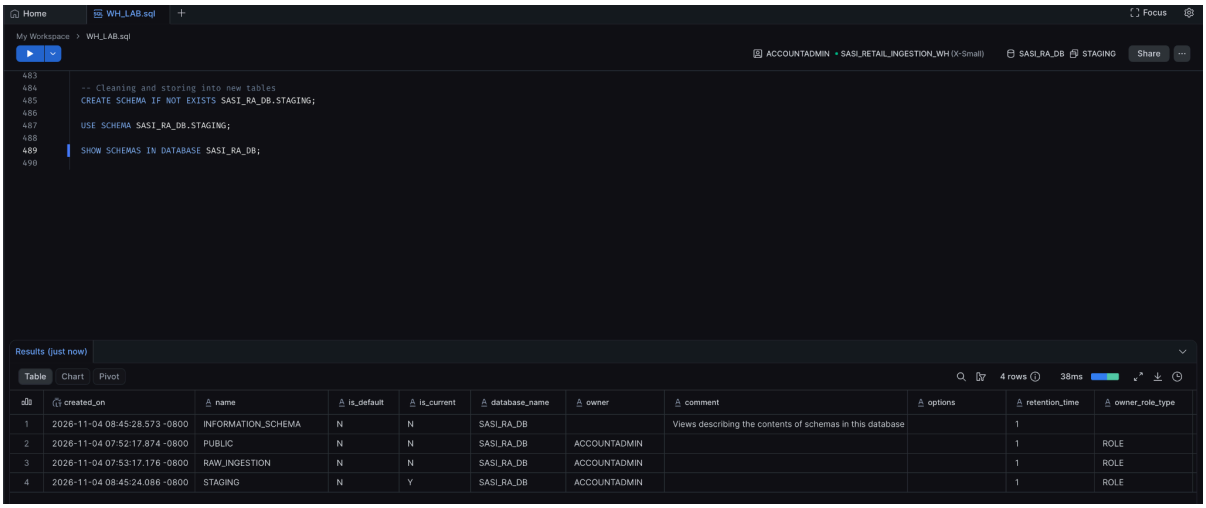
Evidence 38

Schema inventory showing the newly created STAGING schema alongside RAW_INGESTION.



Evidence 39

Additional worksheet evidence supplied for the staging-schema setup and final schema inventory.



REVISION 3 — STAGING TRANSFORMATIONS, ANALYTICS & QUERY OPTIMIZATION

Additional evidence supplied on 05 November 2026. This revision documents the populated STAGING schema, row-count reconciliation, staging-layer quality checks, analytical SQL and a first query-profile comparison. It supplements rather than replaces the raw-ingestion and profiling procedures in the earlier sections.

22. Staging transformation validation

22.1 Customer staging results

The STAGING.STG_CUSTOMERS table contains 121 records and 121 distinct CUSTOMER_ID values. Three rows have a NULL EMAIL, while zero rows have a NULL SIGNUP_DATE. The sample output also shows that the staging transformation retains both the original signup-date text (SIGNUP_DATE_RAW) and a normalized DATE column (SIGNUP_DATE). City values are standardized for reporting—for example, lowercase and mixed-case Curitiba values become the same canonical city label. A DATA_QUALITY_STATUS column marks the three missing-email records as MISSING_EMAIL and the remaining displayed records as VALID.

```
SELECT
    COUNT(*) AS TOTAL_RECORDS,
    COUNT(DISTINCT CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
    COUNT_IF(EMAIL IS NULL) AS MISSING_EMAILS,
    COUNT_IF(SIGNUP_DATE IS NULL) AS INVALID_SIGNUP_DATES
FROM SASI_RA_DB.STAGING.STG_CUSTOMERS;

SELECT *
FROM SASI_RA_DB.STAGING.STG_CUSTOMERS
LIMIT 20;
```

Interpretation: the metric named INVALID_SIGNUP_DATES is implemented as SIGNUP_DATE IS NULL in the captured query. It therefore counts missing/unparsed normalized dates, not every possible date-quality problem. If a separate raw date was present but ambiguous or parsed under the wrong locale convention, that issue requires an explicit validation rule.

22.2 Raw-to-staging reconciliation

The reconciliation query compares raw and staging row counts for both datasets. Customers decrease from 125 raw rows to 121 staged rows (a net reduction of 4); orders decrease from 213 to 198 (a net reduction of 15). This is a useful control point, but the counts alone do not explain why individual records were excluded, merged or rejected. The pipeline should retain row-level rejection reasons and source lineage so that every raw record can be accounted for.

```
SELECT
    'CUSTOMERS' AS DATASET,
    (SELECT COUNT(*) FROM SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS) AS RAW_COUNT,
    (SELECT COUNT(*) FROM SASI_RA_DB.STAGING.STG_CUSTOMERS) AS STAGING_COUNT
UNION ALL
SELECT
    'ORDERS',
    (SELECT COUNT(*) FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS),
    (SELECT COUNT(*) FROM SASI_RA_DB.STAGING.STG_ORDERS);
```

22.3 Staged order quality summary

The staged-order quality query reports 198 total rows, 1 missing ORDER_ID, 1 invalid PURCHASE_DT, 16 missing delivery dates, 2 invalid delivery dates, 10 delivery-before-purchase anomalies, 17 invalid prices and 14 invalid quantities. The screenshot also includes unmatched-customer and revenue-eligible metrics, but their result values are not fully visible; they are therefore not stated here. These issue counts may overlap: a single order can fail more than one rule. They must not be added together and interpreted as a count of distinct rejected rows.

```
SELECT
    COUNT(*) AS TOTAL_ORDERS,
    COUNT_IF(ORDER_ID IS NULL) AS MISSING_ORDER_IDS,
    COUNT_IF(PURCHASE_DT IS NULL) AS INVALID_PURCHASE_DATES,
    COUNT_IF(DELIVERY_STATUS = 'MISSING_DELIVERY_DATE') AS MISSING_DELIVERY_DATES,
    COUNT_IF(DELIVERY_STATUS = 'INVALID_DELIVERY_DATE') AS INVALID_DELIVERY_DATES,
    COUNT_IF(DELIVERY_STATUS = 'DELIVERY_BEFORE_PURCHASE') AS DELIVERY_DATE_ANOMALIES,
    COUNT_IF(PRICE_NUM IS NULL OR PRICE_NUM <= 0) AS INVALID_PRICES,
    COUNT_IF(
        QUANTITY_NUM IS NULL
        OR QUANTITY_NUM <= 0
        OR QUANTITY_NUM <> TRUNC(QUANTITY_NUM)
    ) AS INVALID_QUANTITIES,
    COUNT_IF(CUSTOMER_MATCH_STATUS = 'UNMATCHED_CUSTOMER') AS UNMATCHED_CUSTOMERS,
    COUNT_IF(REVENUE_ELIGIBLE) AS REVENUE_ELIGIBLE_ORDERS
FROM SASI_RA_DB.STAGING.STG_ORDERS;
```

A useful distinction: the 16 missing delivery dates, 2 invalid delivery dates and 10 temporal anomalies are different conditions. Missing means no source date was supplied; invalid means a non-empty value could not be parsed; delivery-before-purchase means both dates were parsed but violate the expected chronology. The same record could appear in more than one quality dimension only where the rules allow it.

23. Analytical data modeling and business queries

23.1 Product-category performance

The category aggregation filters to REVENUE_ELIGIBLE = TRUE and groups by PRODUCT_CATEGORY. It reports distinct orders, units sold, revenue and average line value. Electronics leads revenue at 60,682.58, followed by Sports at 46,753.88 and Furniture at 35,968.68. Across the six displayed categories, the reported revenue sums to 189,719.06. Sports has the highest average order-line value (2,922.12), despite Electronics leading total revenue.

```
SELECT
    PRODUCT_CATEGORY,
    COUNT(DISTINCT ORDER_ID) AS TOTAL_ORDERS,
    SUM(QUANTITY_NUM) AS UNITS_SOLD,
    ROUND(SUM(LINE_TOTAL), 2) AS TOTAL_REVENUE,
    ROUND(AVG(LINE_TOTAL), 2) AS AVG_ORDER_LINE_VALUE
FROM STG_ORDERS
WHERE REVENUE_ELIGIBLE = TRUE
GROUP BY PRODUCT_CATEGORY
ORDER BY TOTAL_REVENUE DESC;
```

23.2 Monthly revenue trend

The monthly query groups eligible order lines by DATE_TRUNC('MONTH', PURCHASE_DT). The result contains ten months, January–October 2024, and totals 170 distinct orders. April 2024 is the strongest month shown at 51,277.50, while June is the weakest at 6,758.60. These values describe the captured dataset and should not be extrapolated as a full-year trend because November and December are absent.

```
SELECT
    DATE_TRUNC('MONTH', PURCHASE_DT) AS SALES_MONTH,
    COUNT(DISTINCT ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(LINE_TOTAL), 2) AS MONTHLY_REVENUE,
    ROUND(AVG(LINE_TOTAL), 2) AS AVG_ORDER_VALUE
FROM STG_ORDERS
WHERE REVENUE_ELIGIBLE = TRUE
GROUP BY DATE_TRUNC('MONTH', PURCHASE_DT)
ORDER BY SALES_MONTH;
```

23.3 Top customers and buying patterns

The top-customer query joins staged customers to eligible staged orders and ranks customers by total spend. The leading displayed customer is CUST251 (Hugo Ferreira) with two orders and 38,841.48 total spend; the reported average line value is 19,420.74. This is a high-value customer result, but it should be checked against the order grain and potential duplicate order identifiers before being used in a KPI.

```
SELECT
    C.CUSTOMER_ID,
    C.CUSTOMER_NAME,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(O.LINE_TOTAL), 2) AS TOTAL_SPENT,
    ROUND(AVG(O.LINE_TOTAL), 2) AS AVG_ORDER_VALUE
FROM STG_CUSTOMERS C
INNER JOIN STG_ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
WHERE O.REVENUE_ELIGIBLE = TRUE
GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
ORDER BY TOTAL_SPENT DESC
LIMIT 10;
```

The customer buying-pattern query uses a LEFT JOIN and places the revenue-eligibility condition in the JOIN predicate. This preserves customer rows even when no eligible order matches. COALESCE converts the resulting NULL revenue to zero, allowing a 121-row customer-oriented output that includes customers with no eligible purchases. This is analytically different from the INNER JOIN top-customer query, which returns only customers with matching eligible orders.

```
SELECT
    C.CUSTOMER_ID,
    C.CUSTOMER_NAME,
    C.CUSTOMER_CITY,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    COALESCE(ROUND(SUM(O.LINE_TOTAL), 2), 0) AS TOTAL_REVENUE
FROM STG_CUSTOMERS C
LEFT JOIN STG_ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
   AND O.REVENUE_ELIGIBLE = TRUE
GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME, C.CUSTOMER_CITY
ORDER BY TOTAL_REVENUE DESC;
```

23.4 City-wise sales and payment-method analysis

The city query uses an INNER JOIN and eligible orders, so its registered-customer count represents customers participating in the matched eligible-order result—not every customer registered in the staging table. Belo Horizonte leads city revenue at 59,533.50, followed by Curitiba at 52,485.38 and São Paulo at 43,694.87. The city query returns 163 orders across the five cities shown.

The payment-method result exposes a standardization issue: CREDIT_CARD and CREDIT CARD appear as separate categories. Their separate totals are 50,128.08 and 42,674.83 respectively. Before building a dashboard, normalize these labels to one canonical payment category (after confirming they mean the same thing). DEBIT CARD leads the displayed payment categories with 51,233.67 revenue.

```
SELECT
    PAYMENT_TYPE,
    COUNT(DISTINCT ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(LINE_TOTAL), 2) AS TOTAL_REVENUE,
    ROUND(AVG(LINE_TOTAL), 2) AS AVG_TRANSACTION_VALUE
FROM STG_ORDERS
WHERE REVENUE_ELIGIBLE = TRUE
GROUP BY PAYMENT_TYPE
ORDER BY TOTAL_REVENUE DESC;
```

24. Query optimization experiment

The experiment disables the session result cache before running the original city-sales aggregation. The optimized version introduces an ELIGIBLE_ORDERS common table expression (CTE) that filters REVENUE_ELIGIBLE = TRUE and projects only CUSTOMER_ID, ORDER_ID and LINE_TOTAL before joining to STG_CUSTOMERS. Both queries return the same five city rows and the same totals, which is the required correctness check before comparing execution behavior.

```
ALTER SESSION SET USE_CACHED_RESULT = FALSE;

-- Optimized form: filter and project the order input first
WITH ELIGIBLE_ORDERS AS (
    SELECT CUSTOMER_ID, ORDER_ID, LINE_TOTAL
    FROM SASI_RA_DB.STAGING.STG_ORDERS
    WHERE REVENUE_ELIGIBLE = TRUE
)
SELECT
    C.CUSTOMER_CITY,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(O.LINE_TOTAL), 2) AS TOTAL_REVENUE
FROM SASI_RA_DB.STAGING.STG_CUSTOMERS C
INNER JOIN ELIGIBLE_ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
GROUP BY C.CUSTOMER_CITY
ORDER BY TOTAL_REVENUE DESC;
```

The worksheet result panes show 354 ms for the original statement and 90 ms for the CTE version. However, the two Query Profile panels show total execution times of 260 ms and 309 ms, respectively. Both profiles report 0.02 MB scanned, 2 partitions scanned out of 2 total, and 170 rows passing the filter. Thus, the available evidence does not show micro-partition pruning or a consistent speedup. The differing elapsed-time figures may reflect measurement scope, scheduling, compilation, or run-to-run variation; do not claim a proven 74% performance improvement from this single small-data test.

Learning note: a CTE can improve readability and make early filtering/projection explicit, but Snowflake may inline and optimize the CTE together with the surrounding statement. SQL text that looks more optimized is not automatically faster. For a defensible benchmark, run both forms repeatedly under comparable warehouse and cache conditions, compare Query Profile operator statistics and bytes/partitions scanned, and use a larger dataset where pruning and join behavior are measurable.

25. Revision 3 findings and follow-up work

- The transformation layer is populated: STG_CUSTOMERS has 121 rows and STG_ORDERS has 198 rows, compared with 125 and 213 raw rows.
- The staging layer reports 3 missing customer emails and 0 NULL normalized signup dates.
- Staged order checks expose missing/invalid IDs and dates, temporal anomalies, invalid prices and quantities; issue counts can overlap.
- Revenue-eligible analysis reports 189,719.06 across six product categories and ten months (January–October 2024).
- Electronics leads category revenue; April is the strongest month in the captured period; Belo Horizonte leads the city result.
- Payment labels CREDIT_CARD and CREDIT CARD remain split and should be normalized before reporting.
- The original and CTE city queries return identical results; the current profile evidence does not prove a performance improvement or partition pruning.

Recommended next implementation steps

- Create a reconciliation ledger that assigns every raw row to accepted, rejected, duplicate or intentionally excluded status, with a reason code and source-row identifier.

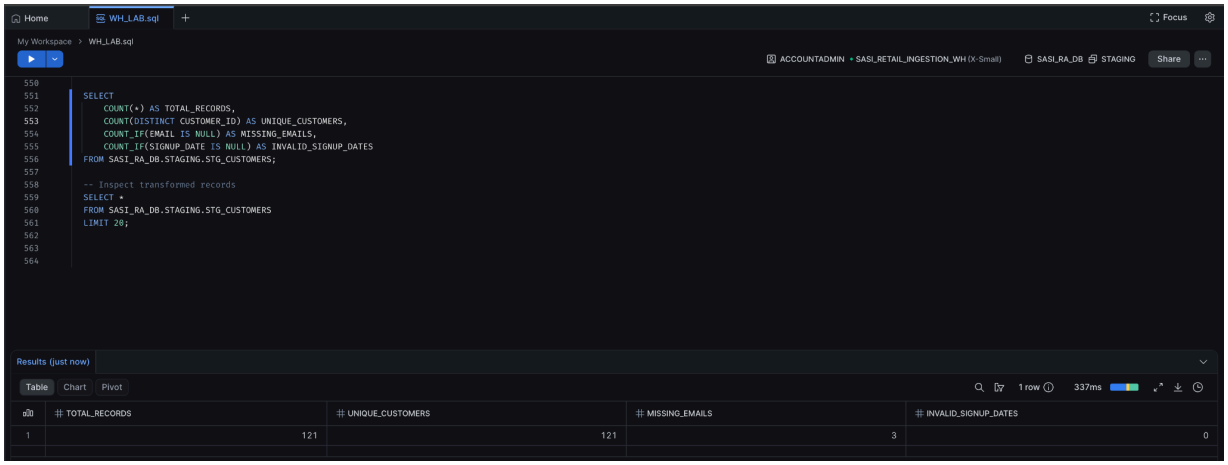
- Verify that the four customer-row and fifteen order-row net differences are fully explained by deterministic transformation rules.
- Document the exact REVENUE_ELIGIBLE definition, currency policy, return/refund treatment and order grain.
- Normalize payment method and other categorical labels with explicit mapping tables rather than ad hoc string replacement.
- Recheck customer/order duplicate keys before using COUNT(DISTINCT ORDER_ID) as a business order metric.
- Build curated dimensional models and Power BI measures only after data grain, relationship keys and quality acceptance rules are stable.
- Repeat query benchmarks and compare operator-level profiles on a materially larger dataset.

Appendix B. Revision 3 screenshot evidence

The following 14 screenshots are embedded in chronological order. They extend the evidence register from Evidence 39 to Evidence 53.

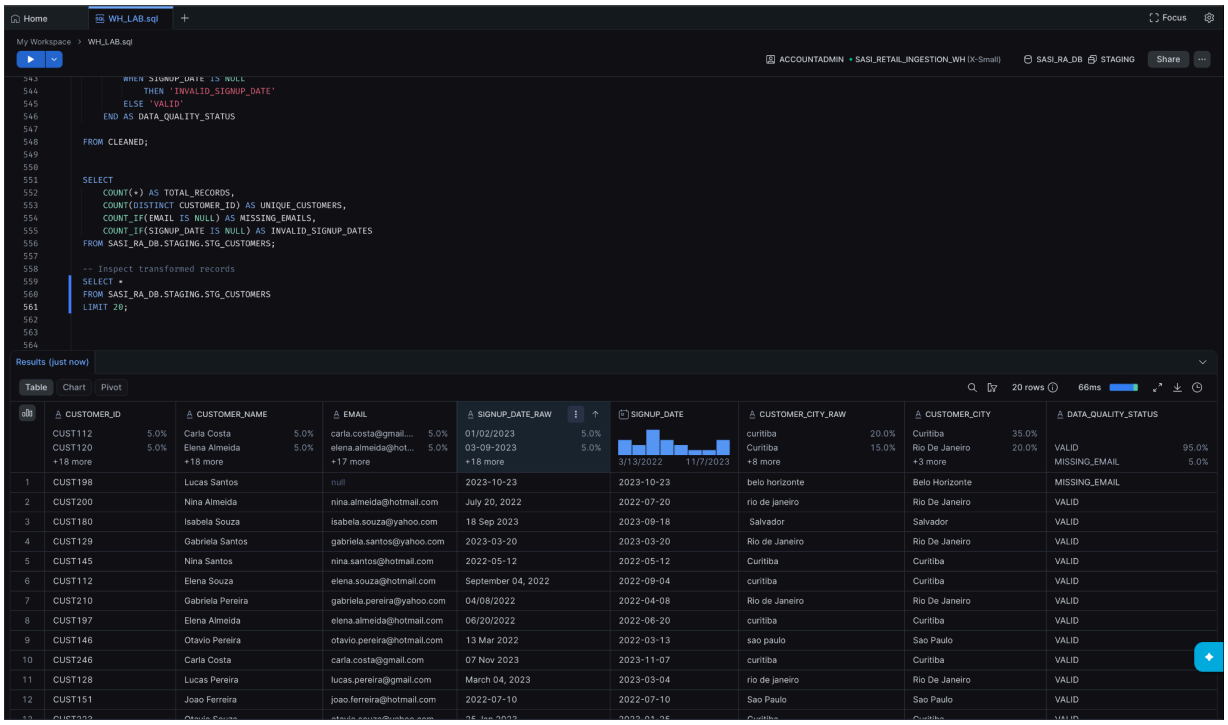
Evidence 40

Customer staging quality summary: 121 rows, 121 distinct customer IDs, 3 missing emails and 0 NULL normalized signup dates.



Evidence 41

Sample of 20 transformed customer records, showing raw and normalized signup dates, standardized city values and DATA_QUALITY_STATUS.



```
        WHEN SIGNUP_DATE IS NULL
            THEN 'INVALID_SIGNUP_DATE'
        ELSE 'VALID'
    END AS DATA_QUALITY_STATUS

FROM CLEANED;

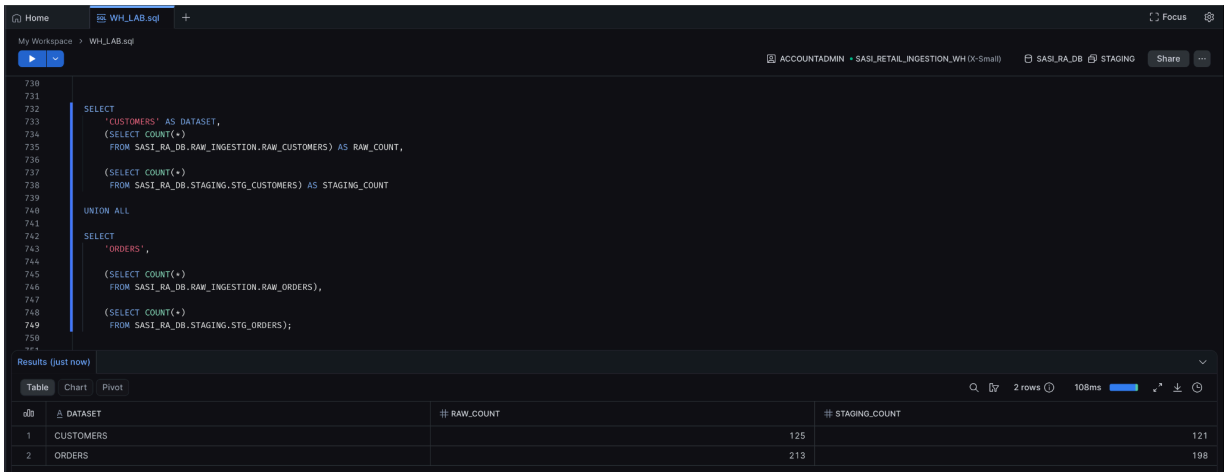

SELECT
    COUNT(*) AS TOTAL_RECORDS,
    COUNT(DISTINCT CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
    COUNT_IF(EMAIL IS NULL) AS MISSING_EMAILS,
    COUNT_IF(SIGNUP_DATE IS NULL) AS INVALID_SIGNUP_DATES
FROM SASI_RA_DB.STAGING.STG_CUSTOMERS;

-- Inspect transformed records
SELECT *
FROM SASI_RA_DB.STAGING.STG_CUSTOMERS
LIMIT 20;
```

	CUSTOMER_ID	CUSTOMER_NAME	EMAIL	SIGNUP_DATE_RAW	SIGNUP_DATE	CUSTOMER_CITY_RAW	CUSTOMER_CITY	DATA_QUALITY_STATUS
	CUST112 5.0% CUST120 5.0% +18 more	Carla Costa 5.0% Elena Almeida 5.0% +18 more	carla.costa@gmail.... 5.0% elena.almeida@hot... 5.0% +17 more	01/02/2023 5.0% 03-09-2023 5.0% +18 more	3/13/2022 11/7/2023	curitiba 20.0% Curitiba 15.0% +8 more	Curitiba 35.0% Rio De Janeiro 20.0% +3 more	VALID 95.0% MISSING_EMAIL 5.0%
1	CUST198	Lucas Santos	null	2023-10-23	2023-10-23	belo horizonte	Belo Horizonte	MISSING_EMAIL
2	CUST200	Nina Almeida	nina.almeida@hotmail.com	July 20, 2022	2022-07-20	rio de janeiro	Rio De Janeiro	VALID
3	CUST180	Isabela Souza	isabela.souza@yahoo.com	18 Sep 2023	2023-09-18	Salvador	Salvador	VALID
4	CUST129	Gabriela Santos	gabriela.santos@yahoo.com	2023-03-20	2023-03-20	Rio de Janeiro	Rio De Janeiro	VALID
5	CUST145	Nina Santos	nina.santos@hotmail.com	2022-05-12	2022-05-12	Curitiba	Curitiba	VALID
6	CUST112	Elena Souza	elena.souza@hotmail.com	September 04, 2022	2022-09-04	curitiba	Curitiba	VALID
7	CUST210	Gabriela Pereira	gabriela.pereira@yahoo.com	04/08/2022	2022-04-08	Rio de Janeiro	Rio De Janeiro	VALID
8	CUST197	Elena Almeida	elena.almeida@hotmail.com	06/20/2022	2022-06-20	curitiba	Curitiba	VALID
9	CUST146	Otavio Pereira	otavio.pereira@hotmail.com	13 Mar 2022	2022-03-13	sao paulo	Sao Paulo	VALID
10	CUST246	Carla Costa	carla.costa@gmail.com	07 Nov 2023	2023-11-07	curitiba	Curitiba	VALID
11	CUST128	Lucas Pereira	lucas.pereira@gmail.com	March 04, 2023	2023-03-04	rio de janeiro	Rio De Janeiro	VALID
12	CUST151	Joao Ferreira	joao.ferreira@hotmail.com	2022-07-10	2022-07-10	Sao Paulo	Sao Paulo	VALID

Evidence 42

Raw-to-staging row-count reconciliation: customers 125 to 121; orders 213 to 198.



```
SELECT
    'CUSTOMERS' AS DATASET,
    (SELECT COUNT(*)
     FROM SASI_RA_DB.RAW_INGESTION.RAW_CUSTOMERS) AS RAW_COUNT,

    (SELECT COUNT(*)
     FROM SASI_RA_DB.STAGING.STG_CUSTOMERS) AS STAGING_COUNT

UNION ALL

SELECT
    'ORDERS',

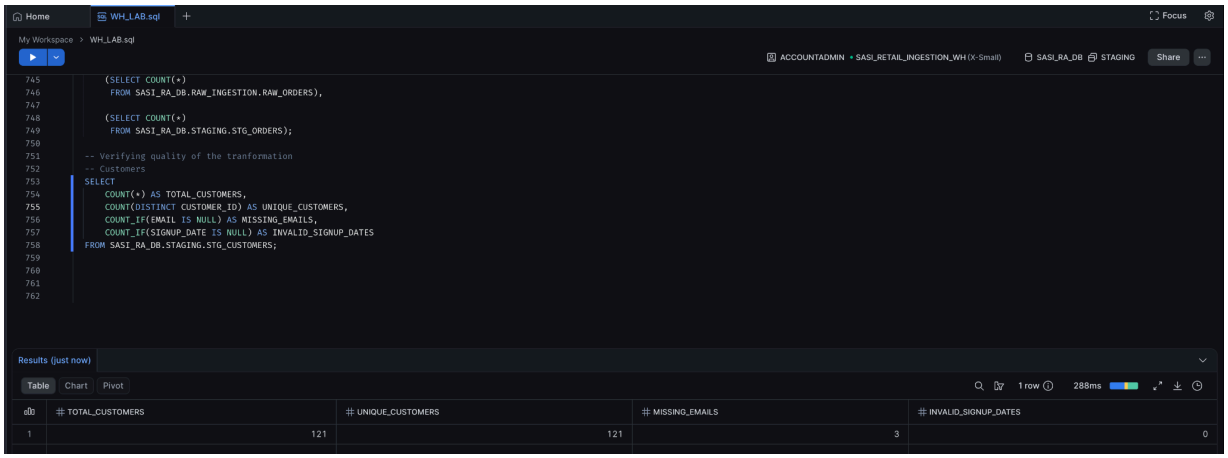
    (SELECT COUNT(*)
     FROM SASI_RA_DB.RAW_INGESTION.RAW_ORDERS),

    (SELECT COUNT(*)
     FROM SASI_RA_DB.STAGING.STG_ORDERS);
```

	DATASET	RAW_COUNT	STAGING_COUNT
1	CUSTOMERS	125	121
2	ORDERS	213	198

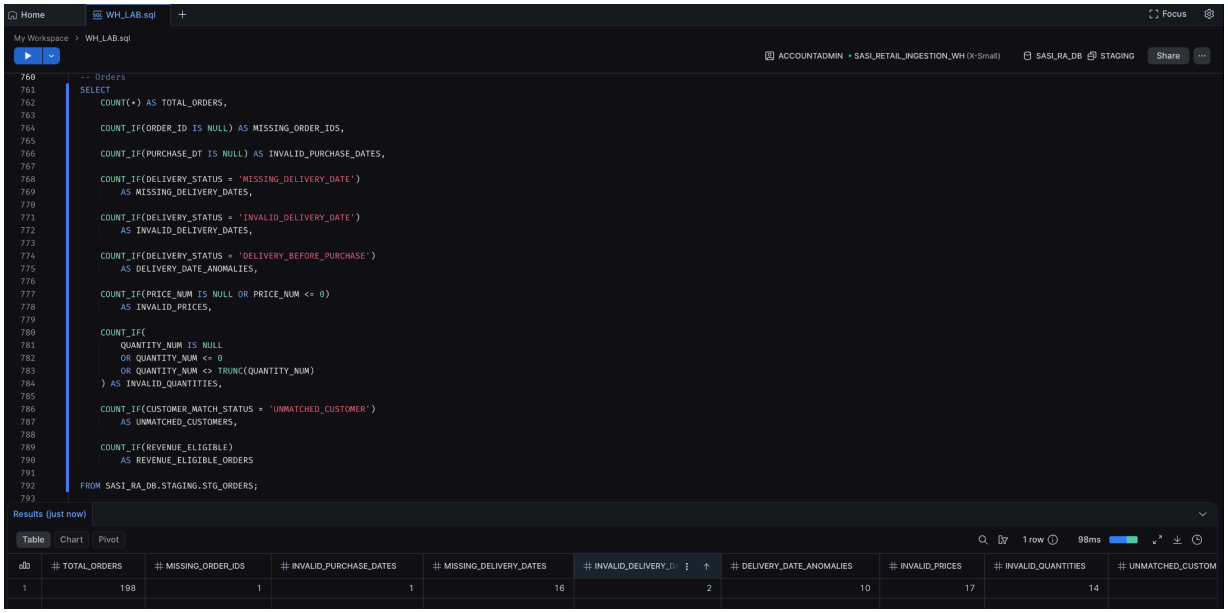
Evidence 43

Customer staging validation repeated as a focused result: 121 total and unique customers, 3 missing emails and 0 NULL signup dates.



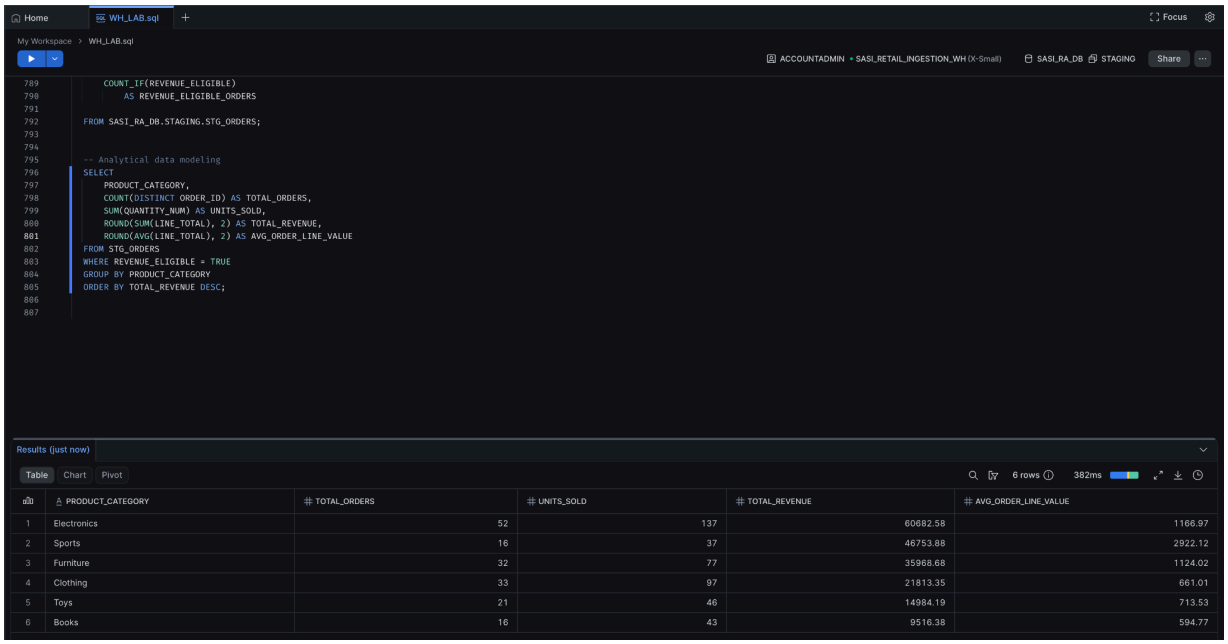
Evidence 44

Staged-order quality summary: 198 rows with missing/invalid identifiers and dates, temporal anomalies, invalid prices and quantities.



Evidence 45

Product-category performance for revenue-eligible orders, including distinct orders, units sold, revenue and average line value.



```
    COUNT_IF(REVENUE_ELIGIBLE)
        AS REVENUE_ELIGIBLE_ORDERS

FROM SASI_RA_DB.STAGING.STG_ORDERS;

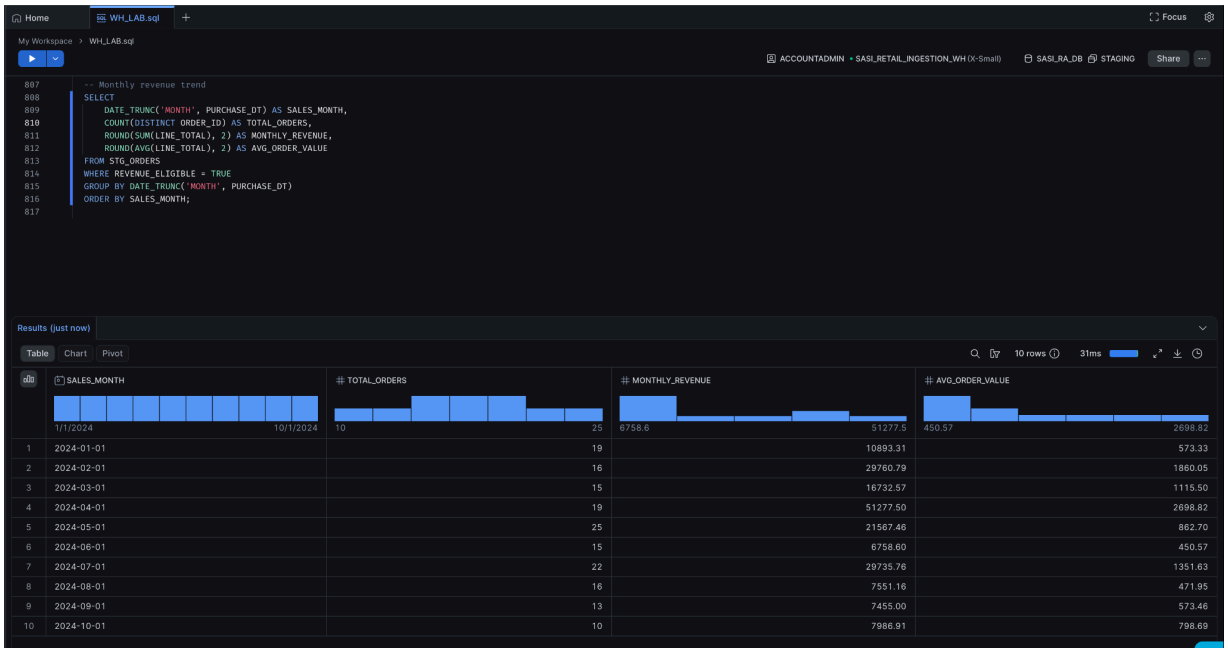

-- Analytical data modeling
SELECT
    PRODUCT_CATEGORY,
    COUNT(DISTINCT ORDER_ID) AS TOTAL_ORDERS,
    SUM(QUANTITY_NUM) AS UNITS_SOLD,
    ROUND(SUM(LINE_TOTAL), 2) AS TOTAL_REVENUE,
    ROUND(AVG(LINE_TOTAL), 2) AS AVG_ORDER_LINE_VALUE
FROM STG_ORDERS
WHERE REVENUE_ELIGIBLE = TRUE
GROUP BY PRODUCT_CATEGORY
ORDER BY TOTAL_REVENUE DESC;
```

Results (just now) — Table | Chart | Pivot — 6 rows — 382ms

	PRODUCT_CATEGORY	TOTAL_ORDERS	UNITS_SOLD	TOTAL_REVENUE	AVG_ORDER_LINE_VALUE
1	Electronics	52	137	60682.58	1166.97
2	Sports	16	37	46753.88	2922.12
3	Furniture	32	77	35968.68	1124.02
4	Clothing	33	97	21813.35	661.01
5	Toys	21	46	14984.19	713.53
6	Books	16	43	9516.38	594.77

Evidence 46

Monthly revenue trend for January–October 2024, including order counts, monthly revenue and average order value.



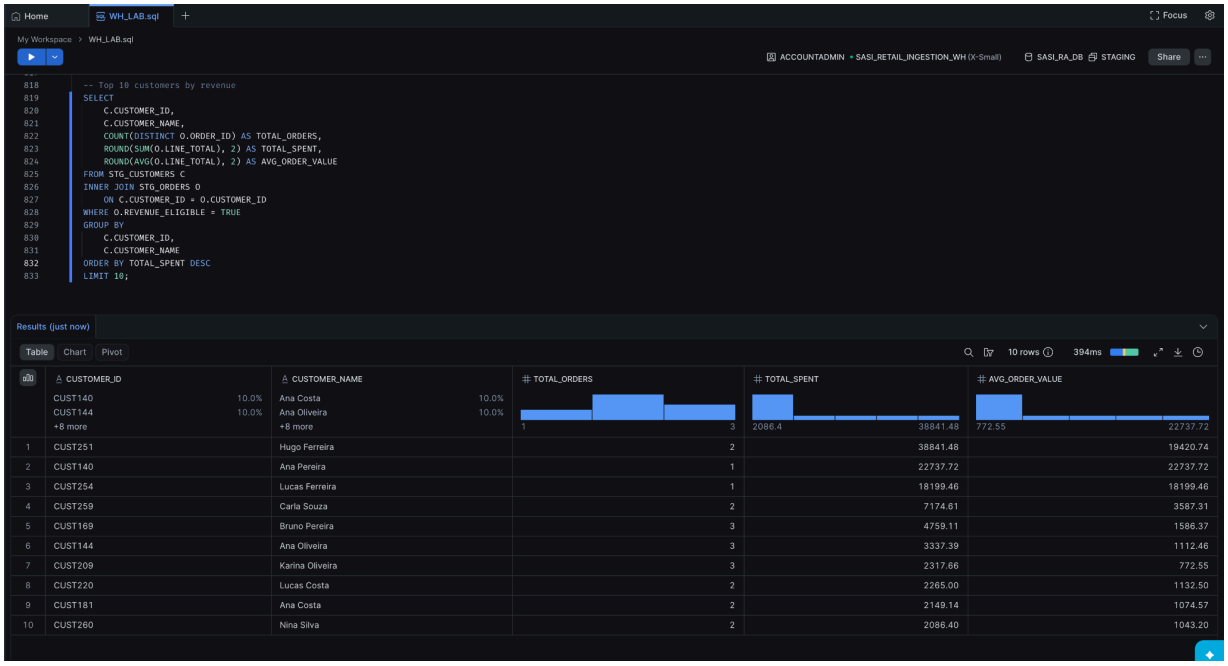
```
-- Monthly revenue trend
SELECT
    DATE_TRUNC('MONTH', PURCHASE_DT) AS SALES_MONTH,
    COUNT(DISTINCT ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(LINE_TOTAL), 2) AS MONTHLY_REVENUE,
    ROUND(AVG(LINE_TOTAL), 2) AS AVG_ORDER_VALUE
FROM STG_ORDERS
WHERE REVENUE_ELIGIBLE = TRUE
GROUP BY DATE_TRUNC('MONTH', PURCHASE_DT)
ORDER BY SALES_MONTH;
```

Results (just now) — Table | Chart | Pivot — 10 rows — 31ms

	SALES_MONTH	TOTAL_ORDERS	MONTHLY_REVENUE	AVG_ORDER_VALUE
	1/1/2024 – 10/1/2024	10 – 25	6758.6 – 51277.5	450.57 – 2698.82
1	2024-01-01	19	10893.31	573.33
2	2024-02-01	16	29760.79	1860.05
3	2024-03-01	15	16732.57	1115.50
4	2024-04-01	19	51277.50	2698.82
5	2024-05-01	25	21567.46	862.70
6	2024-06-01	15	6758.60	450.57
7	2024-07-01	22	29735.76	1351.63
8	2024-08-01	16	7551.16	471.95
9	2024-09-01	13	7455.00	573.46
10	2024-10-01	10	7986.91	798.69

Evidence 47

Top 10 customers by eligible-order spend, led by CUST251 with 38,841.48 across two orders.



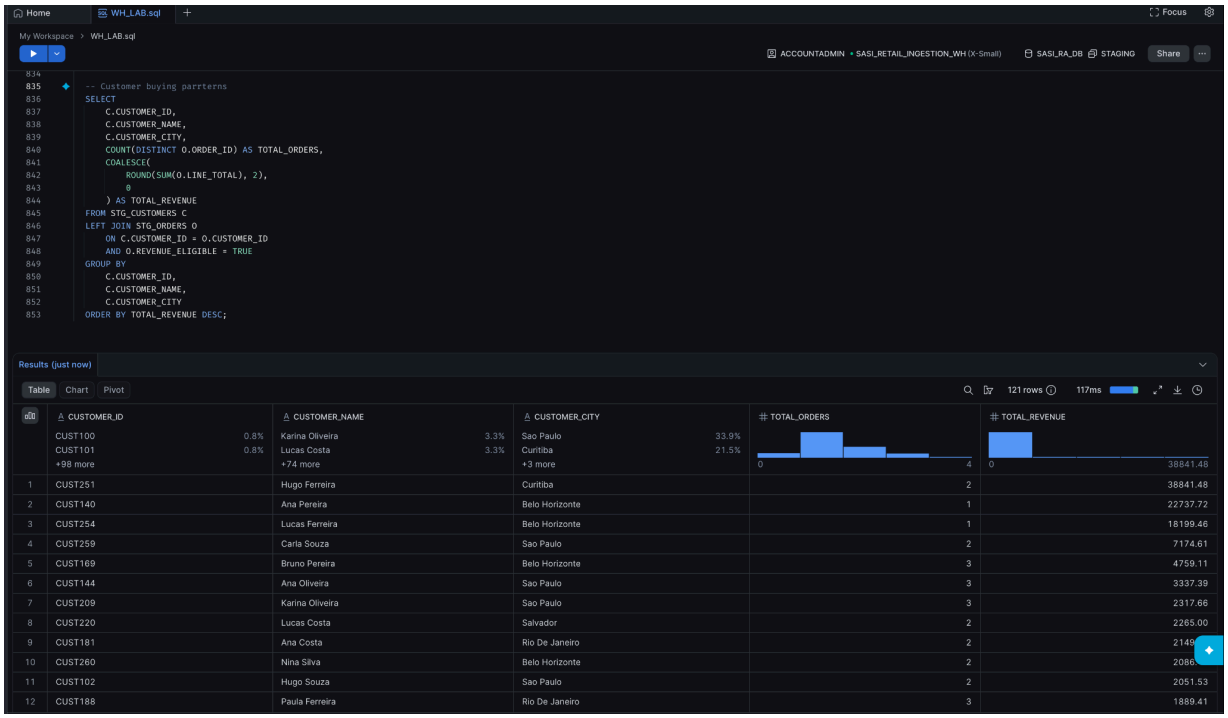
```
-- Top 10 customers by revenue
SELECT
    C.CUSTOMER_ID,
    C.CUSTOMER_NAME,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(O.LINE_TOTAL), 2) AS TOTAL_SPENT,
    ROUND(AVG(O.LINE_TOTAL), 2) AS AVG_ORDER_VALUE
FROM STG_CUSTOMERS C
INNER JOIN STG_ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
WHERE O.REVENUE_ELIGIBLE = TRUE
GROUP BY
    C.CUSTOMER_ID,
    C.CUSTOMER_NAME
ORDER BY TOTAL_SPENT DESC
LIMIT 10;
```

Results (just now) — Table | Chart | Pivot — 10 rows — 394ms

	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_SPENT	AVG_ORDER_VALUE
	CUST140 10.0% CUST144 10.0% +8 more	Ana Costa 10.0% Ana Oliveira 10.0% +8 more	1 – 3	2086.4 – 38841.48	772.55 – 22737.72
1	CUST251	Hugo Ferreira	2	38841.48	19420.74
2	CUST140	Ana Pereira	1	22737.72	22737.72
3	CUST254	Lucas Ferreira	1	18199.46	18199.46
4	CUST259	Carla Souza	2	7174.61	3587.31
5	CUST169	Bruno Pereira	3	4759.11	1586.37
6	CUST144	Ana Oliveira	3	3337.39	1112.46
7	CUST209	Karina Oliveira	3	2317.66	772.55
8	CUST220	Lucas Costa	2	2265.00	1132.50
9	CUST181	Ana Costa	2	2149.14	1074.57
10	CUST260	Nina Silva	2	2086.40	1043.20

Evidence 48

Customer buying-pattern output using LEFT JOIN and COALESCE, retaining all 121 staged customer records.



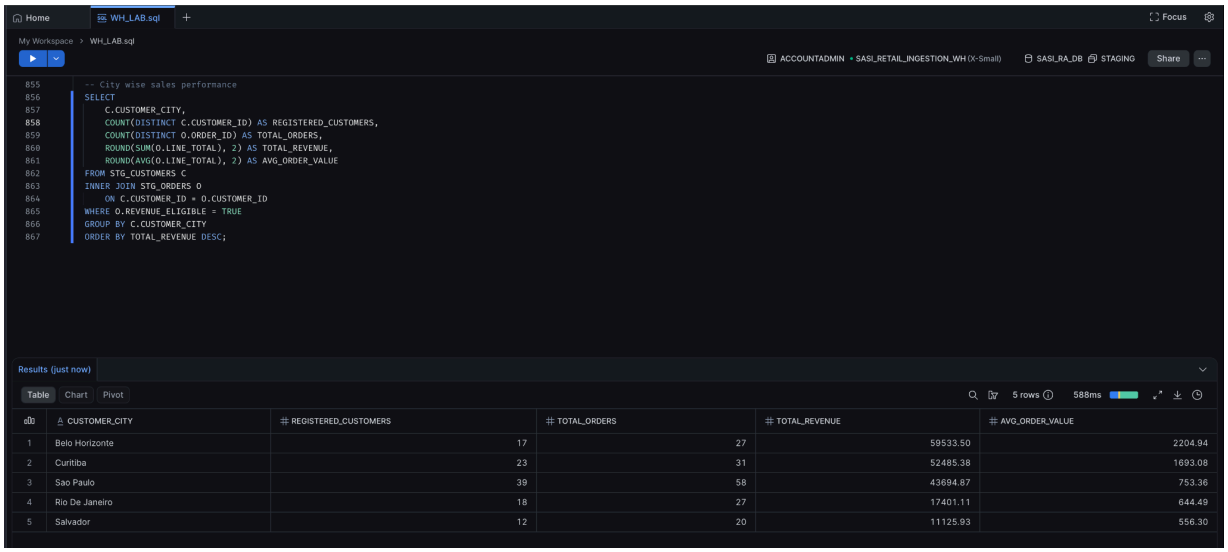
```
-- Customer buying parrterns
SELECT
    C.CUSTOMER_ID,
    C.CUSTOMER_NAME,
    C.CUSTOMER_CITY,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    COALESCE(
        ROUND(SUM(O.LINE_TOTAL), 2),
        0
    ) AS TOTAL_REVENUE
FROM STG_CUSTOMERS C
LEFT JOIN STG_ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
    AND O.REVENUE_ELIGIBLE = TRUE
GROUP BY
    C.CUSTOMER_ID,
    C.CUSTOMER_NAME,
    C.CUSTOMER_CITY
ORDER BY TOTAL_REVENUE DESC;
```

Results (just now) — Table | Chart | Pivot — 121 rows — 117ms

	CUSTOMER_ID	CUSTOMER_NAME	CUSTOMER_CITY	TOTAL_ORDERS	TOTAL_REVENUE
	CUST100 0.8% CUST101 0.8% +98 more	Karina Oliveira 3.3% Lucas Costa 3.3% +74 more	Sao Paulo 33.9% Curitiba 21.5% +3 more	0 – 4	0 – 38841.48
1	CUST251	Hugo Ferreira	Curitiba	2	38841.48
2	CUST140	Ana Pereira	Belo Horizonte	1	22737.72
3	CUST254	Lucas Ferreira	Belo Horizonte	1	18199.46
4	CUST259	Carla Souza	Sao Paulo	2	7174.61
5	CUST169	Bruno Pereira	Belo Horizonte	3	4759.11
6	CUST144	Ana Oliveira	Sao Paulo	3	3337.39
7	CUST209	Karina Oliveira	Sao Paulo	3	2317.66
8	CUST220	Lucas Costa	Salvador	2	2265.00
9	CUST181	Ana Costa	Rio De Janeiro	2	2149
10	CUST260	Nina Silva	Belo Horizonte	2	2086
11	CUST102	Hugo Souza	Sao Paulo	2	2051.53
12	CUST188	Paula Ferreira	Rio De Janeiro	3	1889.41

Evidence 49

City-wise sales performance across five cities, with matched registered customers, eligible orders, revenue and average order value.

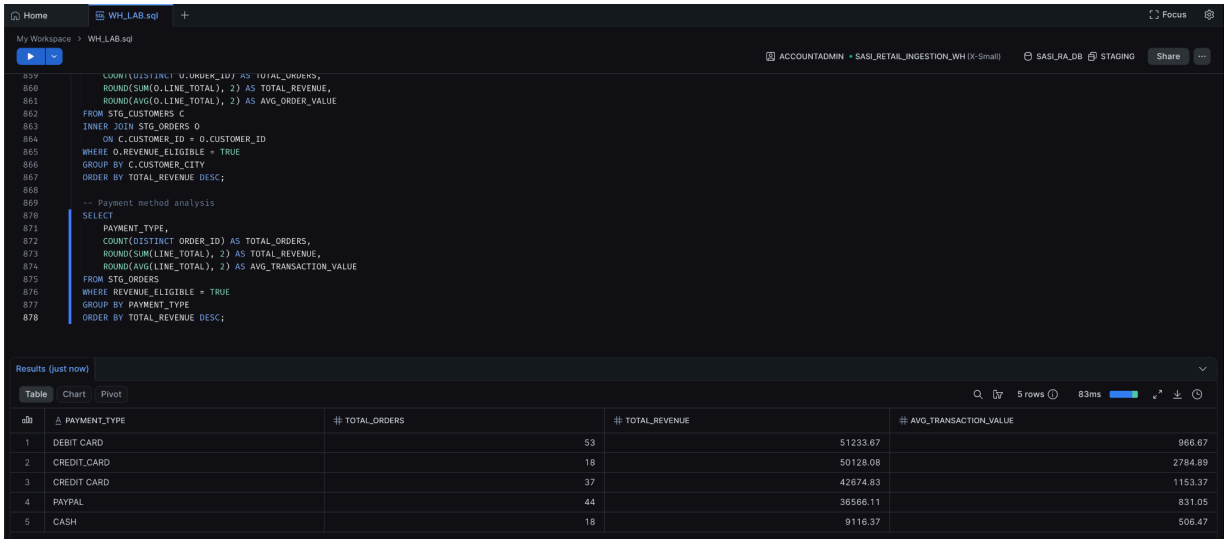


```
-- City wise sales performance
SELECT
    C.CUSTOMER_CITY,
    COUNT(DISTINCT C.CUSTOMER_ID) AS REGISTERED_CUSTOMERS,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(O.LINE_TOTAL), 2) AS TOTAL_REVENUE,
    ROUND(AVG(O.LINE_TOTAL), 2) AS AVG_ORDER_VALUE
FROM STG_CUSTOMERS C
INNER JOIN STG_ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
WHERE O.REVENUE_ELIGIBLE = TRUE
GROUP BY C.CUSTOMER_CITY
ORDER BY TOTAL_REVENUE DESC;
```

	CUSTOMER_CITY	REGISTERED_CUSTOMERS	TOTAL_ORDERS	TOTAL_REVENUE	AVG_ORDER_VALUE
1	Belo Horizonte	17	27	59533.50	2204.94
2	Curitiba	23	31	52485.38	1693.08
3	Sao Paulo	39	58	43694.87	753.36
4	Rio De Janeiro	18	27	17401.11	644.49
5	Salvador	12	20	11125.93	556.30

Evidence 50

Payment-method revenue analysis; CREDIT_CARD and CREDIT CARD are still represented as separate categories.



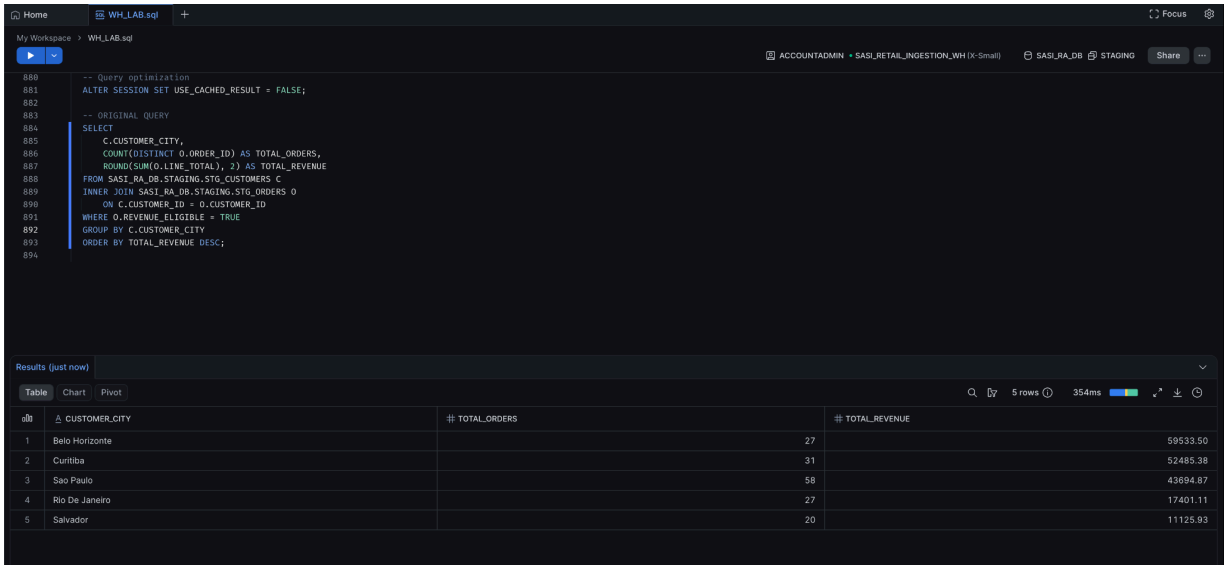
```
    ROUND(SUM(O.LINE_TOTAL), 2) AS TOTAL_REVENUE,
    ROUND(AVG(O.LINE_TOTAL), 2) AS AVG_ORDER_VALUE
FROM STG_CUSTOMERS C
INNER JOIN STG_ORDERS O
    ON C.CUSTOMER_ID = O.CUSTOMER_ID
WHERE O.REVENUE_ELIGIBLE = TRUE
GROUP BY C.CUSTOMER_CITY
ORDER BY TOTAL_REVENUE DESC;

-- Payment method analysis
SELECT
    PAYMENT_TYPE,
    COUNT(DISTINCT ORDER_ID) AS TOTAL_ORDERS,
    ROUND(SUM(LINE_TOTAL), 2) AS TOTAL_REVENUE,
    ROUND(AVG(LINE_TOTAL), 2) AS AVG_TRANSACTION_VALUE
FROM STG_ORDERS
WHERE REVENUE_ELIGIBLE = TRUE
GROUP BY PAYMENT_TYPE
ORDER BY TOTAL_REVENUE DESC;
```

	PAYMENT_TYPE	TOTAL_ORDERS	TOTAL_REVENUE	AVG_TRANSACTION_VALUE
1	DEBIT CARD	53	51233.67	966.67
2	CREDIT_CARD	18	50128.08	2784.89
3	CREDIT CARD	37	42674.83	1153.37
4	PAYPAL	44	36566.11	831.05
5	CASH	18	9116.37	506.47

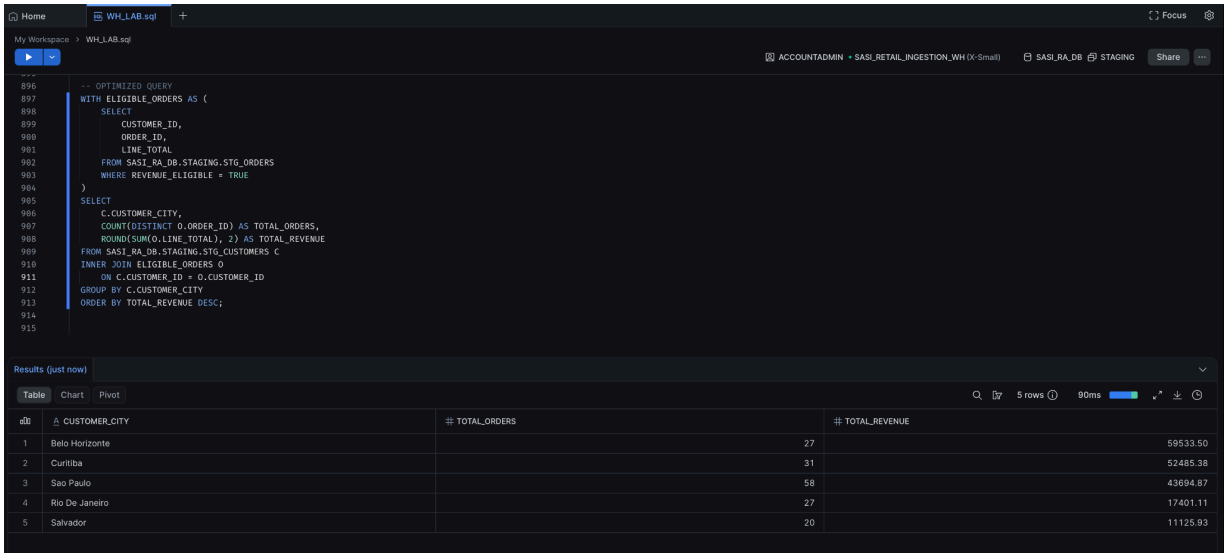
Evidence 51

Original city-sales aggregation with result pane showing approximately 354 ms.



Evidence 52

CTE-based city-sales query with the same five result rows and approximately 90 ms shown in the worksheet.



Evidence 53

Side-by-side Query Profiles: both scan 2 of 2 partitions and 0.02 MB; displayed execution times are 260 ms and 309 ms.

